RÉPUBLIQUE TUNISIENNE
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
Université de Carthage
Institut National des Sciences Appliquées et de Technologie

DevSecOps

Spécialité : Génie Logiciel
Niveau : 5e année

TP2 – CI/CD sécurisé

Intégration de la Sécurité dans un Pipeline CI/CD

Sécurisation d'une application web Flask conteneurisée
Bandit, pip-audit, OWASP Dependency-Check, OWASP ZAP et GitHub Actions

Présenté par :

Yassine Dahmoul
Mohamed Sahbi Ben Kalaia
Mohamed Dhia Medini
Takoua Ayadi

Année Universitaire : 2026 – 2027

Table des matières

1 Contexte et Objectifs **2**

2 Environnement Technique **3**

3 Partie I – Pipeline de base : compiler et tester **3**
3.1 Structure du projet . . . 3
3.2 Application et tests . . . 4
3.3 Étude du fichier `.github/workflows/ci.yml` . . . 5
3.4 Les deux jobs de la phase 1 . . . 5
3.5 Construction progressive du pipeline . . . 6

4 Partie II – SAST et SCA avec seuils bloquants **6**
4.1 Politique de blocage . . . 6
4.2 SAST – Bandit . . . 7
4.3 SCA – pip-audit . . . 7
4.4 SCA – OWASP Dependency-Check . . . 8
4.5 Lisibilité des résultats . . . 8
4.6 Un constat réel : une dépendance du TP1 devenue vulnérable . . . 8

5 Partie III – DAST sur un environnement éphémère **9**
5.1 Pourquoi un DAST . . . 9
5.2 L'environnement éphémère . . . 9
5.3 Le plan ZAP (Automation Framework) . . . 10
5.4 Règle de blocage . . . 11
5.5 Résultats sur l'application saine . . . 11
5.6 Un faux positif rencontré : triage et filtre ciblé . . . 12

6 Partie IV – Porte de sécurité, protection de la branche et publication **15**
6.1 Le job `security-gate` . . . 15
6.2 Protection de la branche `main` . . . 16
6.3 Le job `release` : ne jamais publier une version non sécurisée . . . 17

7 Partie V – Validation : pull request vulnérable, puis corrigée **18**
7.1 Scénario . . . 18
7.2 Pull request n° 1 : la version vulnérable est bloquée . . . 18
7.3 Pull request n° 2 : la version corrigée passe . . . 28
7.4 Publication de l'image . . . 32
7.5 Synthèse comparée . . . 32

8 Conclusion **32**

Annexe A – Fichier ci.yml **33**

Annexe B – Plan ZAP **37**

Annexe C – Index des preuves **39**

1 Contexte et Objectifs

Au TP1, les scans de sécurité étaient lancés *à la main*, une seule fois. Rien n'oblige quelqu'un à les relancer après chaque modification, chaque nouveau commit peut introduire une faille, et de nouvelles vulnérabilités sont publiées chaque jour sur des bibliothèques qui n'ont pas changé. Ce TP automatise ces contrôles dans un **pipeline CI/CD** qui joue le rôle de **garde-fou** (*guardrail*) : à chaque modification, les mêmes contrôles s'exécutent automatiquement, et leur résultat **décide** de ce qui peut être fusionné et publié.

Trois familles de contrôles, complémentaires, sont intégrées au pipeline :

Approche	**Ce qui est analysé**	**Outil utilisé**
SAST (*Static Application Security Testing*)	Notre code source, sans l'exécuter	Bandit
SCA (*Software Composition Analysis*)	Les bibliothèques tierces (directes et transitives)	pip-audit, OWASP Dependency-Check
DAST (*Dynamic Application Security Testing*)	L'application *en cours d'exécution*, attaquée de l'extérieur	OWASP ZAP

Objectifs principaux (sujet du TP) :

— Automatiser les tests de sécurité dans un processus de développement continu.

— Comprendre le rôle d'un pipeline CI/CD comme **garde-fou de sécurité**.

— Configurer des **règles de validation** qui bloquent les déploiements non sécurisés.

Le travail suit les quatre activités du sujet :

Activité	**Réalisation**
1. Pipeline de base	Compilation, tests unitaires et construction de l'image Docker (Partie I)
2. SAST et SCA	Bandit, pip-audit et Dependency-Check avec des seuils bloquants (Partie II)
3. DAST	Scan OWASP ZAP authentifié sur un environnement éphémère (Partie III) Porte de sécurité, protection de la branche et publication conditionnelle (Partie IV)
4. Validation	Une pull request volontairement vulnérable (pipeline en échec), puis une pull request corrigée (pipeline vert) (Partie V)

2 Environnement Technique

Élément	Détail
Plateforme CI/CD	GitHub Actions (exécuteurs `ubuntu-latest`)
Dépôt	`YassineDHML/tp2-devsecops-cicd-pipeline`
Application	*Notes App* (Python 3.13, Flask), version corrigée du TP1
Tests	pytest 9.2.0, pytest-cov 7.1.0
SAST	Bandit 1.9.4
SCA	pip-audit 2.10.1 (base PyPA / OSV) ; OWASP Dependency-Check 13.0.0 (base NVD, image Docker)
DAST	OWASP ZAP 2.17.0 (image Docker `ghcr.io/zaproxy/zaproxy:2.17.0`), *Automation Framework*
Conteneurisation	Docker : image de l'application, réseau et conteneurs éphémères du DAST
Registre d'images	GitHub Container Registry (`ghcr.io`)

Les images des scanners sont épinglées par version, et toutes les actions GitHub par **empreinte de commit** : un tag peut être déplacé par un attaquant qui prend le contrôle d'une action, pas une empreinte.

3 Partie I – Pipeline de base : compiler et tester

3.1 Structure du projet

```
tp2-devsecops-cicd-pipeline/
 .github/workflows/ci.yml      # le pipeline (un seul fichier)
 app/                          # application Flask issue du TP1
 tests/                        # tests de l'application et du script de gate
 scripts/
   dast.sh                     # environnement ephemere, scan ZAP, gate,
    destruction
   security_report.py          # resumes des rapports et regle de blocage du DAST
 zap/automation.yaml           # plan ZAP : scan authentifie
 reports/                      # preuves d'execution (PR vulnerable / PR corrigee
    )
 Dockerfile  requirements.txt  requirements-dev.txt
 RAPPORT.md                    # rapport complet au format Markdown
```

Le dépôt est public sur GitHub (Figure 1) : dossiers du projet, dernier commit validé par le pipeline et lien vers la pull request ouverte.

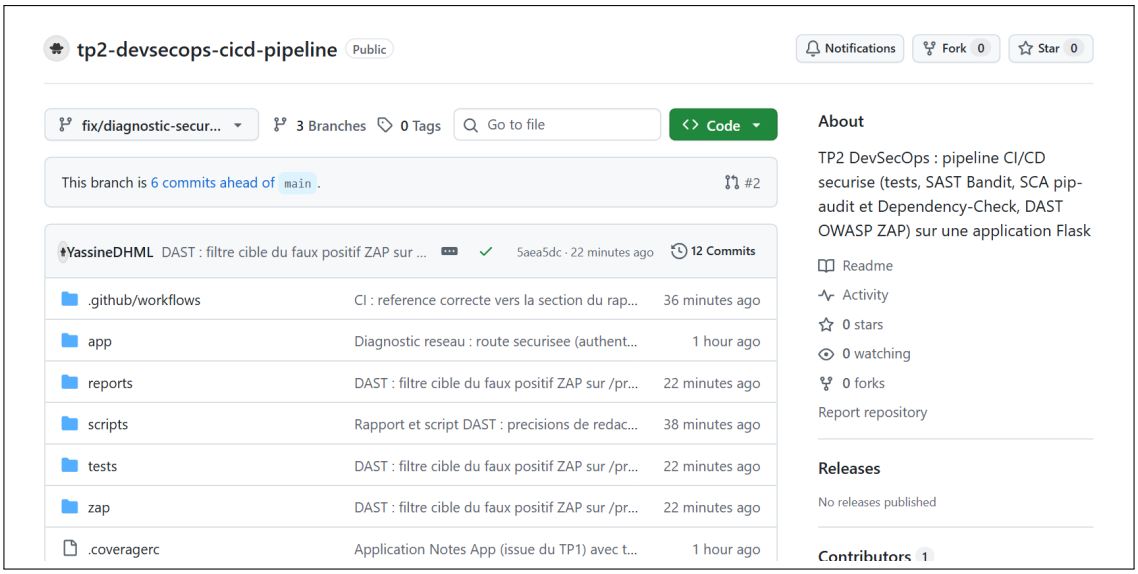


FIGURE 1 – Dépôt GitHub (branche de la pull request corrigée) : fichiers du projet et dernier commit validé.

3.2 Application et tests

L'application est la version corrigée du TP1 (*Notes App* : comptes utilisateurs, notes, recherche, import YAML, préférences, deux outils réseau). Pour que le pipeline ait quelque chose à vérifier, une suite `pytest` a été ajoutée (client de test Flask, base SQLite temporaire pour chaque test) :

— **comportement fonctionnel** : inscription, connexion, déconnexion, notes, recherche, import, préférences ;

— **propriétés de sécurité corrigées au TP1** : requêtes paramétrées, échappement HTML, mots de passe hachés, cookie `HttpOnly` / `SameSite`, YAML sûr, ping sans shell, protection anti-SSRF, en-têtes de sécurité. Si l'une de ces corrections était annulée par erreur, un test échouerait ;

— **script de gate** : `tests/test_security_report.py` vérifie la règle de blocage elle-même, car un garde-fou défaillant est pire que pas de garde-fou.

Au total, **79 tests** (dont 14 pour le script de gate) et **99 %** de couverture (lignes et branches) de `app/app.py`. Le pipeline exige 90 % (Figure 2).

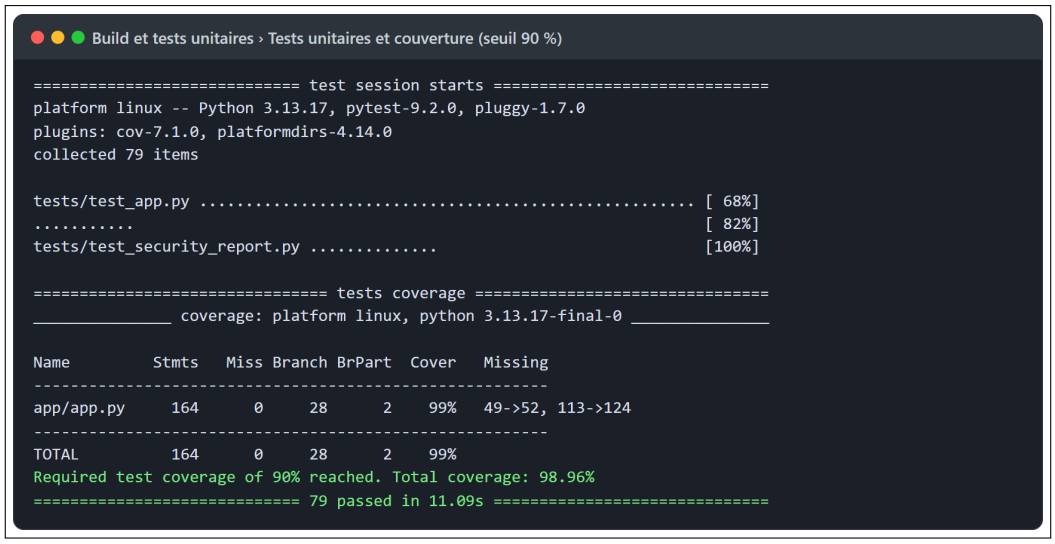


FIGURE 2 – Étape « Tests unitaires et couverture » du job *Build et tests unitaires* : 79 tests réussis, couverture de 98,96 %.

3.3 Étude du fichier `.github/workflows/ci.yml`

Bloc du fichier	Rôle
`on: pull_request, push, schedule, workflow_dispatch`	Contrôle chaque PR **avant** la fusion (c'est là que le garde-fou agit), le code fusionné sur `main`, et une fois par semaine pour détecter les nouvelles CVE sans nouveau commit
`permissions: contents: read`	Moindre privilège : le jeton ne peut que lire le dépôt. Seul le job `release` obtient `packages: write`
`concurrency`	Une nouvelle poussée sur une PR annule l'exécution précédente. Jamais sur `main` : une publication en cours n'est pas interrompue
`env`	Seuils de blocage (`BANDIT_FAIL_LEVEL`, `DC_FAIL_ON_CVSS`) définis une seule fois
`uses: actions/...@<empreinte>`	Actions épinglées par empreinte de commit (protection de la chaîne d'approvisionnement)
`timeout-minutes`	Un job bloqué ne consomme pas de minutes indéfiniment
`if: always()`	Exécute l'étape même après un échec : publication des rapports, destruction de l'environnement éphémère
`needs`	Ordonne les jobs : le DAST attend l'image, la release attend la porte de sécurité

TABLE 1 – Lecture commentée du fichier de configuration du pipeline (fichier complet en annexe)

3.4 Les deux jobs de la phase 1

`build-test`. Python n'est pas compilé, mais deux étapes jouent ce rôle : `python -m compileall` compile en byte-code **tous** les fichiers, y compris ceux qu'aucun test n'importe, et `pytest --cov --cov-fail-under=90` exécute les tests et échoue si la couverture est

insuffisante. Les rapports JUnit et de couverture sont publiés en artefacts.

`build-image`. `docker build` construit **le livrable**, ce qui vérifie que le `Dockerfile` et `requirements.txt` sont cohérents. L'image est étiquetée avec le SHA du commit, exportée (`docker save`) et publiée comme artefact d'une journée. Le DAST la teste et la release la publie : **c'est exactement la même image**, pas une reconstruction.

3.5 Construction progressive du pipeline

L'historique de `main` suit les activités du sujet. Chaque commit qui contient un pipeline est validé par ses propres contrôles : 2 contrôles pour le pipeline de base, 5 avec le SAST et la SCA, 8 avec le DAST, la porte de sécurité et la publication (Figure 3).

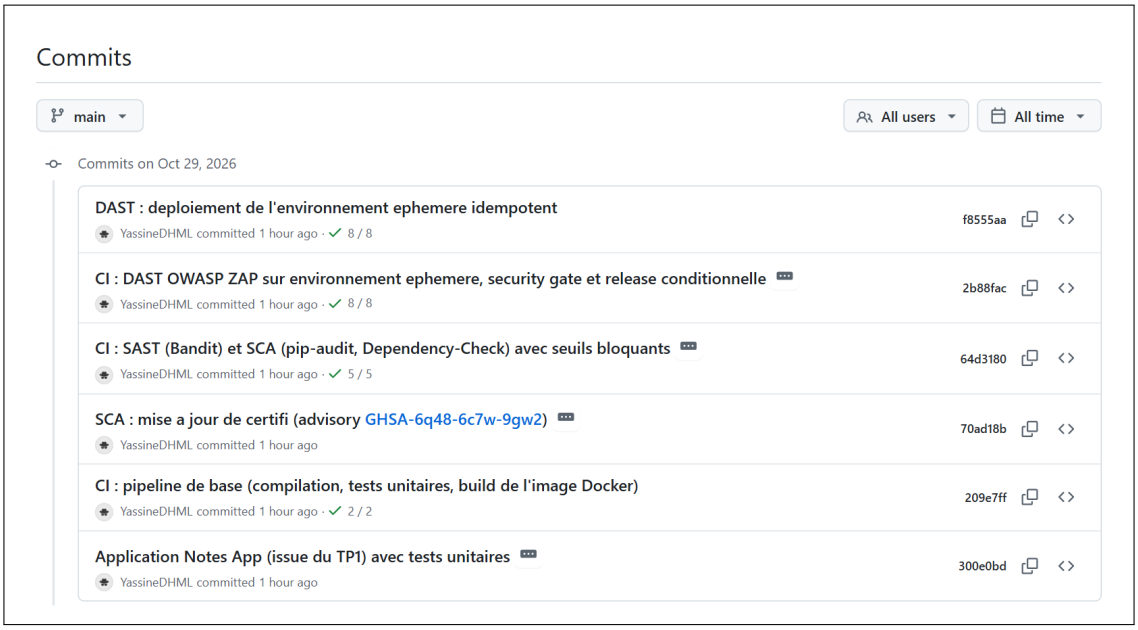


FIGURE 3 – Historique de `main` : le nombre de contrôles réussis par commit passe de 2 à 5 puis à 8.

4 Partie II – SAST et SCA avec seuils bloquants

4.1 Politique de blocage

Le sujet demande de bloquer le pipeline en cas de vulnérabilité **critique ou de haute sévérité**. Chaque outil a sa propre échelle ; le tableau montre comment la règle est traduite.

Outil	Échelle native	Seuil de blocage	Mécanisme
Bandit (SAST)	Low, Medium, High (et confiance)	**High**	`bandit --severity-level high` : code de sortie 1
pip-audit (SCA)	Pas de score CVSS dans la base PyPA / OSV	**Toute vulnérabilité connue**	Code de sortie 1
Dependency-Check (SCA)	CVSS : Medium $< 7 \leq$ High $< 9 \leq$ Critical	**CVSS** $\geq$ **7**	`--failOnCVSS 7` : code de sortie 15
OWASP ZAP (DAST)	Informational, Low, Medium, High (pas de *Critical*)	**High**, le niveau le plus élevé	`security_report.py zap --fail-on high`

Table 2 – Traduction de la règle « critique ou haute sévérité » pour chaque outil

Les constatations de niveau inférieur ne bloquent pas, mais elles sont **affichées** dans l'onglet *Summary* du run et conservées dans les rapports. Pour pip-audit, sans score CVSS, la règle est plus stricte que celle du sujet : toute vulnérabilité connue bloque, ce qui est peu pénalisant car l'outil indique toujours la version corrigée.

4.2 SAST – Bandit

Le travail est fait en deux temps : d'abord les rapports **complets**, tous niveaux confondus (`--exit-zero` : cette étape ne bloque jamais), puis le **gate**, qui relance Bandit avec le seuil et échoue si une constatation l'atteint. Les rapports sont ainsi toujours produits, et le journal du gate affiche directement le code fautif.

Listing 1 – Étapes SAST du pipeline

```
- name: Rapports complets (tous niveaux, non bloquants)
  run: |
    bandit -r app -f json -o ci-reports/bandit.json --exit-zero
    bandit -r app -f html -o ci-reports/bandit.html --exit-zero
    python scripts/security_report.py bandit ci-reports/bandit.json
- name: "GATE SAST : echec si une vulnerabilite de severite HIGH est detectee"
  run: bandit -r app --severity-level "$BANDIT_FAIL_LEVEL"
```

Sur l'application saine, Bandit signale 2 constatations *Low* (`B404`, import de `subprocess`, et `B603`, appel sans shell), analysées et acceptées au TP1 : le gate passe.

4.3 SCA – pip-audit

pip-audit interroge la base **PyPA / OSV** (avis GitHub, PyPI) sur les versions figées de `requirements.txt`, dépendances transitives comprises, et indique la **version corrigée**.

Listing 2 – Étapes pip-audit du pipeline

```
- name: Rapport JSON et resume (non bloquants)
  run: |
    pip-audit -r requirements.txt --no-deps --disable-pip -f json -o ci-reports/
   pip-audit.json || true
    python scripts/security_report.py pip-audit ci-reports/pip-audit.json
- name: "GATE SCA (1/2) : echec si une dependance a une vulnerabilite connue"
  run: pip-audit -r requirements.txt --no-deps --disable-pip --desc off
```

4.4 SCA – OWASP Dependency-Check

Listing 3 – Lancement d'OWASP Dependency-Check dans le pipeline

```
docker run --rm \
  -v "$PWD:/src:ro" \
  -v "$HOME/dc-data:/usr/share/dependency-check/data" \
  -v "$PWD/ci-reports/dependency-check:/report" \
  owasp/dependency-check:13.0.0 \
  --project "TP2 Notes App" --scan /src \
  --exclude "**/.git/**" --exclude "**/ci-reports/**" \
  --enableExperimental --disableOssIndex \
  --nvdDatafeed "https://dependency-check.github.io/DependencyCheck_Builder/
    nvd_cache/" \
  --format HTML --format JSON --out /report \
  --failOnCVSS "$DC_FAIL_ON_CVSS"
```

— `--failOnCVSS 7` : **règle de blocage**, code de sortie 15 dès qu'une CVE a un score supérieur ou égal à 7.

— `--enableExperimental` : l'analyseur de `requirements.txt` est expérimental ; sans cette option, aucune dépendance Python n'est détectée (constaté au TP1).

— `--nvdDatafeed` : miroir officiel de la base NVD (le téléchargement par l'API est très lent sans clé). La base est conservée d'un run à l'autre avec `actions/cache`, sous une clé **hebdomadaire**.

— Le code est monté en **lecture seule** (`:ro`) et l'image est épinglée en version `13.0.0`.

Sur l'application saine : 14 dépendances identifiées dans `requirements.txt`, **0 CVE**.

4.5 Lisibilité des résultats

Le script `scripts/security_report.py` lit le rapport JSON de chaque outil et en écrit un résumé (comptes par niveau, détail des constatations) dans l'onglet *Summary* du run. Chaque job publie aussi ses rapports complets (JSON et HTML) comme **artefacts**. Pour le DAST, c'est ce script qui applique la règle de blocage ; il est donc testé, y compris le cas « rapport absent ou vide » : le gate **échoue fermé**, car sans rapport exploitable la vérification ne peut pas être validée.

4.6 Un constat réel : une dépendance du TP1 devenue vulnérable

En préparant ce TP, pip-audit a été lancé *en local* sur le `requirements.txt` du TP1, sans aucune modification du code (Figure 4).

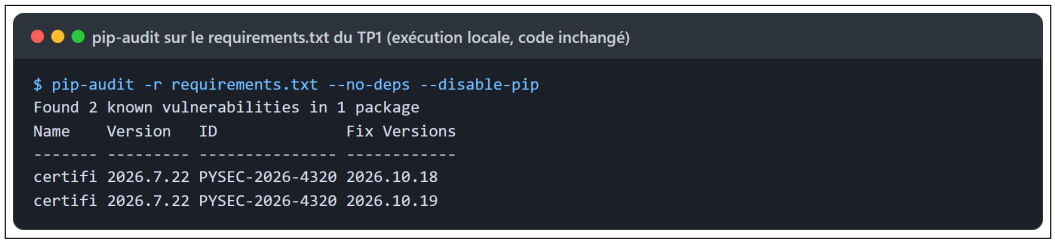


FIGURE 4 – pip-audit sur le `requirements.txt` du TP1 : `certifi` 2026.7.22 est devenu vulnérable.

L'avis (GHSA-6q48-6c7w-9gw2) a été publié le **20 octobre 2026**, après le TP1 : les versions de `certifi` antérieures à 2026.10.19 contiennent des certificats racine *emSign* dont des

certificats intermédiaires ont des clés privées compromises. La dépendance a été mise à jour (`certifi==2026.10.23`). Deux enseignements :

1. **Le code n'a pas changé, mais le risque, si.** C'est la raison d'être de l'exécution planifiée du pipeline : un projet « terminé » doit continuer à être contrôlé.
2. **Les outils SCA ne voient pas tous la même chose.** Dependency-Check, lancé sur la même version, a rapporté **0 vulnérabilité** : son jeu de données (NVD) ne référence que les CVE, or cet avis n'avait pas encore d'identifiant CVE. pip-audit s'appuie sur la base PyPA / OSV, qui reçoit les avis GitHub avant l'attribution d'une CVE. D'où l'intérêt de **combiner deux sources**.

5 Partie III – DAST sur un environnement éphémère

5.1 Pourquoi un DAST

Le SAST lit le code ; le DAST **attaque l'application qui tourne**, comme le ferait un attaquant, sans connaître le code. Il voit ce que le SAST ne peut pas voir : la configuration réelle (en-têtes, cookies), l'effet d'une entrée sur la page produite, ce qu'une route répond sans authentification. Le prix : il est plus lent et il faut une application déployée.

5.2 L'environnement éphémère

« Éphémère » signifie créé pour un seul scan, **identique à chaque fois**, et détruit ensuite. Le script `scripts/dast.sh` en pilote le cycle de vie ; le workflow appelle chaque étape séparément pour qu'elles soient visibles dans l'interface (Figure 5).

Étape du job `dast`	Commande	Ce qui se passe
Charger l'image	`docker load`	Récupère l'image construite par `build-image` : c'est le livrable, pas une reconstruction
Déployer	`dast.sh deploy`	Crée un réseau Docker dédié, lance le conteneur applicatif, attend qu'il réponde, crée un compte de test
Scanner	`dast.sh scan`	Lance ZAP dans un second conteneur du même réseau (cible `http://app:5000`)
Gate	`dast.sh gate`	Lit le rapport ZAP, échoue s'il contient une alerte High
Détruire	`dast.sh destroy`	Supprime conteneur et réseau, même si le scan a échoué (`if: always()`)

Table 3 – Cycle de vie de l'environnement éphémère

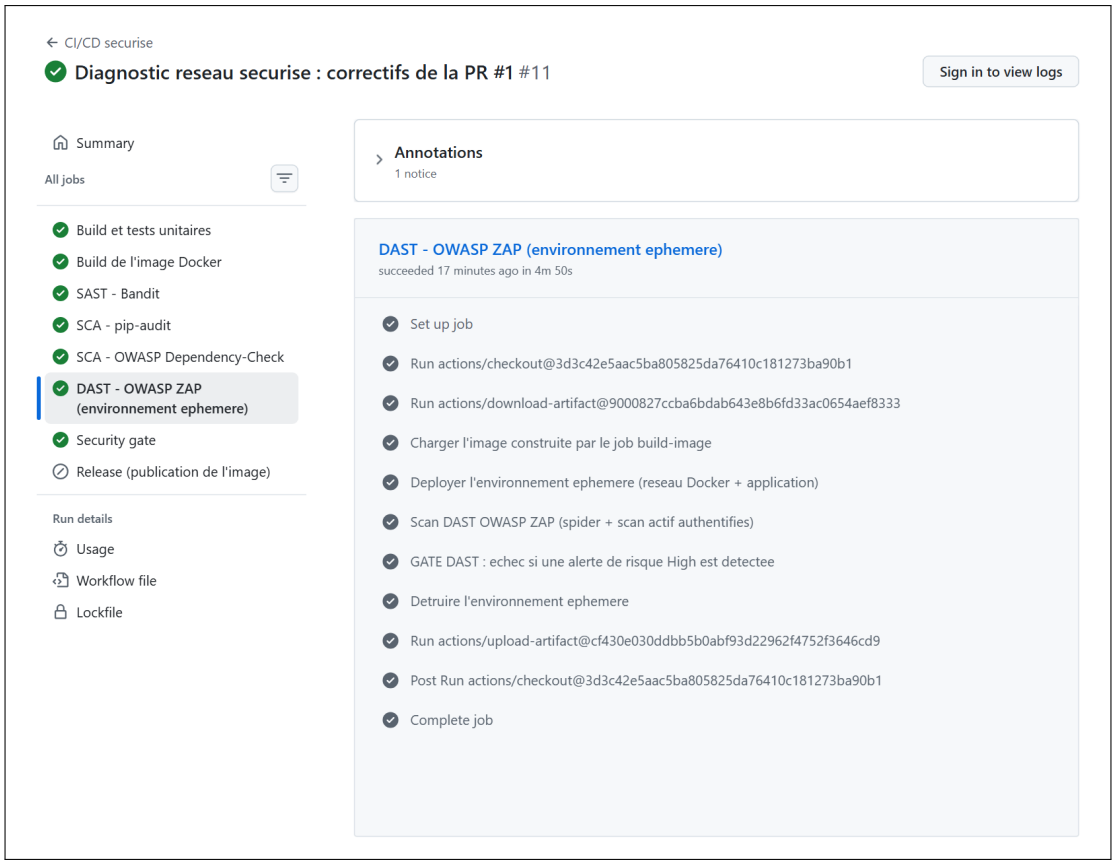


FIGURE 5 – Job *DAST – OWASP ZAP* : chargement de l'image, déploiement, scan, gate et destruction de l'environnement.

Propriétés de sécurité de cet environnement :

Propriété	**Mise en œuvre**
Isolation	Réseau Docker dédié ; le port de l'application n'est publié que sur `127.0.0.1` du runner
Secrets jetables	`SECRET_KEY` et mot de passe du compte de test générés avec `openssl rand -hex` à chaque exécution, stockés dans un fichier en mode `0600` supprimé à la fin ; **rien dans le dépôt ni dans l'image**
État vierge	Base SQLite créée dans le conteneur et détruite avec lui
Pas de compte par défaut	`ADMIN_PASSWORD` n'est pas défini
Processus non privilégié	Le conteneur s'exécute avec l'utilisateur `appuser` (durcissement du TP1)
Machine jetable	Le job tourne sur un exécuteur GitHub neuf, détruit à la fin

5.3 Le plan ZAP (Automation Framework)

Un fichier YAML décrit le scan, ce qui le rend versionné, relisible et reproductible (`zap/automation.yaml`, en annexe).

Élément du plan	Rôle
`contexts`	Périmètre du scan. `/logout` est **exclu** : sinon le spider suivrait le lien « Déconnexion » et fermerait sa propre session
`authentication: form`	ZAP se connecte via `/login` avec le compte jetable (variables d'environnement `ZAP_USER` et `ZAP_PASSWORD`) ; la session est reconnue par la présence du lien « Deconnexion »
`alertFilter`	Requalifie en « faux positif » **une** alerte précise, après triage (voir §5.6)
`requestor`	Déclare `/tools/ping` et `/tools/fetch`, que **aucune page HTML ne référence** : sans cela, le spider ne les découvrirait jamais
`spider` + test `stats`	Explore l'application **authentifiée**. **Garde-fou de couverture** : le job échoue si moins de 8 URL sont découvertes (13 à 14 observées), pour qu'un scan vide ne soit jamais validé
`activeScan`	**Attaque** : injections SQL et de commande, XSS, traversée de chemin… Politique par défaut, durée maximale de 12 minutes
`report` (2 fois)	Rapport JSON (lu par le gate) et rapport HTML (lu par un humain)

TABLE 4 – Les éléments du plan ZAP

Un scan **authentifié** est important : presque toute l'application est derrière la connexion. La Figure 6 montre le journal du plan : les deux routes déclarées, les 14 URL découvertes, le garde-fou de couverture satisfait, puis la fin du scan actif.

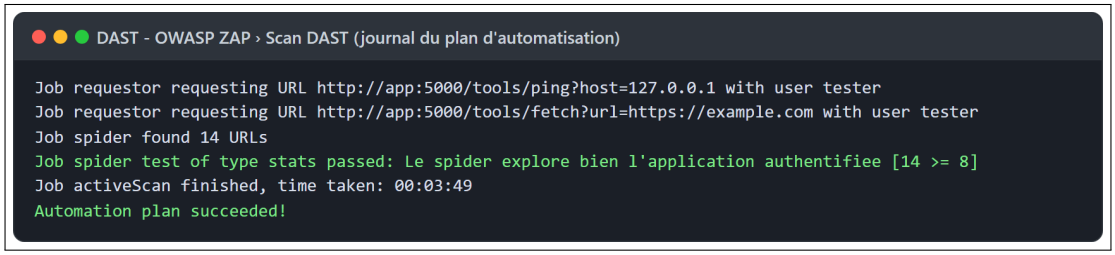


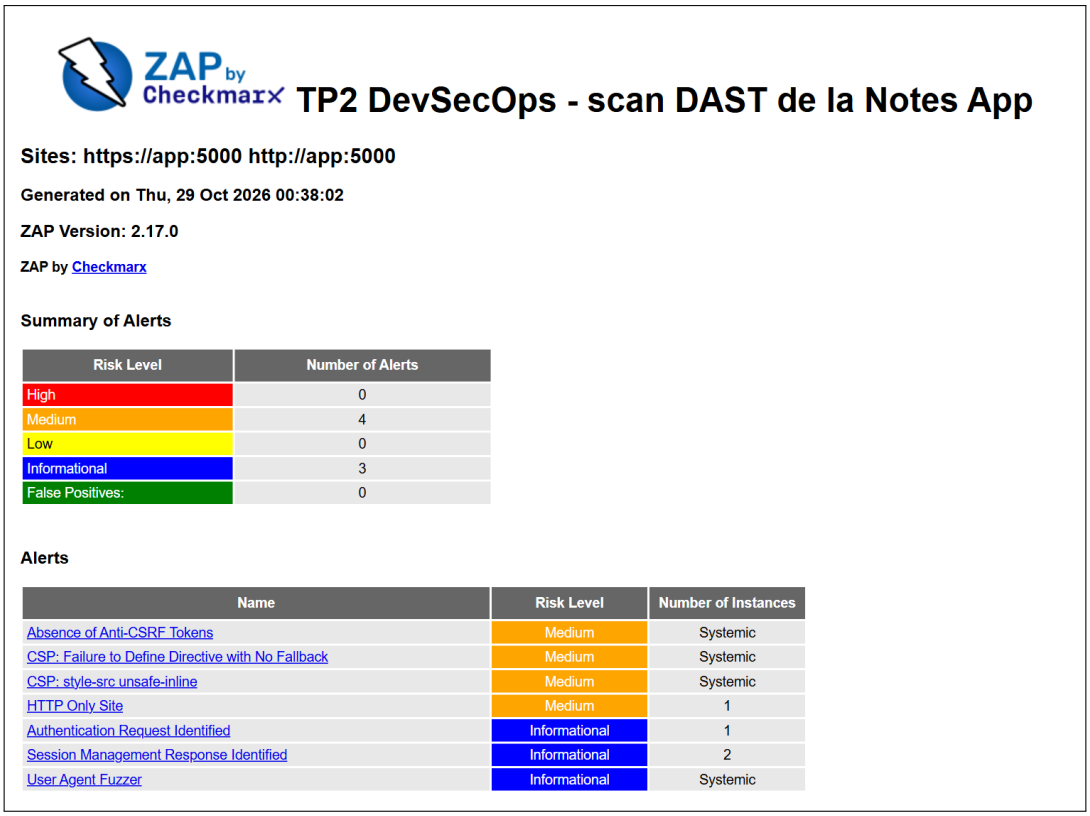
FIGURE 6 – Journal du plan ZAP (PR corrigée) : spider, garde-fou de couverture, scan actif.

5.4 Règle de blocage

ZAP classe ses alertes en *Informational*, *Low*, *Medium* et *High* ; il n'a pas de niveau *Critical*. **High est donc le niveau « critique »** du DAST, et c'est celui qui bloque. ZAP n'ayant pas d'option équivalente à `--failOnCVSS`, la règle est appliquée par `security_report.py zap --fail-on high` sur le rapport JSON. Le script échoue **fermé** : rapport absent, illisible ou sans site analysé, il renvoie le code 2 et le job échoue.

5.5 Résultats sur l'application saine

Le scan de l'application saine donne **0 alerte High**, 4 Medium et 3 Informational (Figure 7) : le gate passe. Les alertes Medium ne bloquent pas mais sont de vraies observations, à analyser (triage) :



ZAP by Checkmarx

TP2 DevSecOps - scan DAST de la Notes App

Sites: https://app:5000 http://app:5000

Generated on Thu, 29 Oct 2026 00:38:02

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	4
Low	0
Informational	3
False Positives:	0

Alerts

Name	Risk Level	Number of Instances
Absence of Anti-CSRF Tokens	Medium	Systemic
CSP: Failure to Define Directive with No Fallback	Medium	Systemic
CSP: style-src unsafe-inline	Medium	Systemic
HTTP Only Site	Medium	1
Authentication Request Identified	Informational	1
Session Management Response Identified	Informational	2
User Agent Fuzzer	Informational	Systemic

FIGURE 7 – Rapport ZAP de l'application corrigée : aucune alerte High.

Alerte (niveau)	**CWE**	**Analyse**
Absence of Anti-CSRF Tokens (Medium)	352	**Vraie faiblesse** : pas de jeton CSRF dans les formulaires. Le cookie `SameSite=Lax` limite le risque. Correctif : Flask-WTF
CSP : Failure to Define Directive with No Fallback (Medium)	693	`frame-ancestors` et `form-action` n'héritent pas de `default-src`. Correctif : les ajouter à la politique
CSP : style-src unsafe-inline (Medium)	693	Le CSS est dans une balise `<style>`. Correctif : fichier statique, retrait de `'unsafe-inline'`
HTTP Only Site (Medium)	311	**Attendu** : l'environnement éphémère sert l'application en HTTP. En production : HTTPS, HSTS, cookie `Secure`
3 alertes Informational	–	Informations de ZAP sur son propre fonctionnement

TABLE 5 – Triage des alertes de niveau Medium (hors périmètre du TP, laissées en l'état)

5.6 Un faux positif rencontré : triage et filtre ciblé

Pendant la validation, un run de la PR corrigée a été **bloqué par le DAST**, alors que le code applicatif n'avait pas changé depuis le run précédent, entièrement vert : le commit ne modifiait que la documentation (Figures 8 et 9).

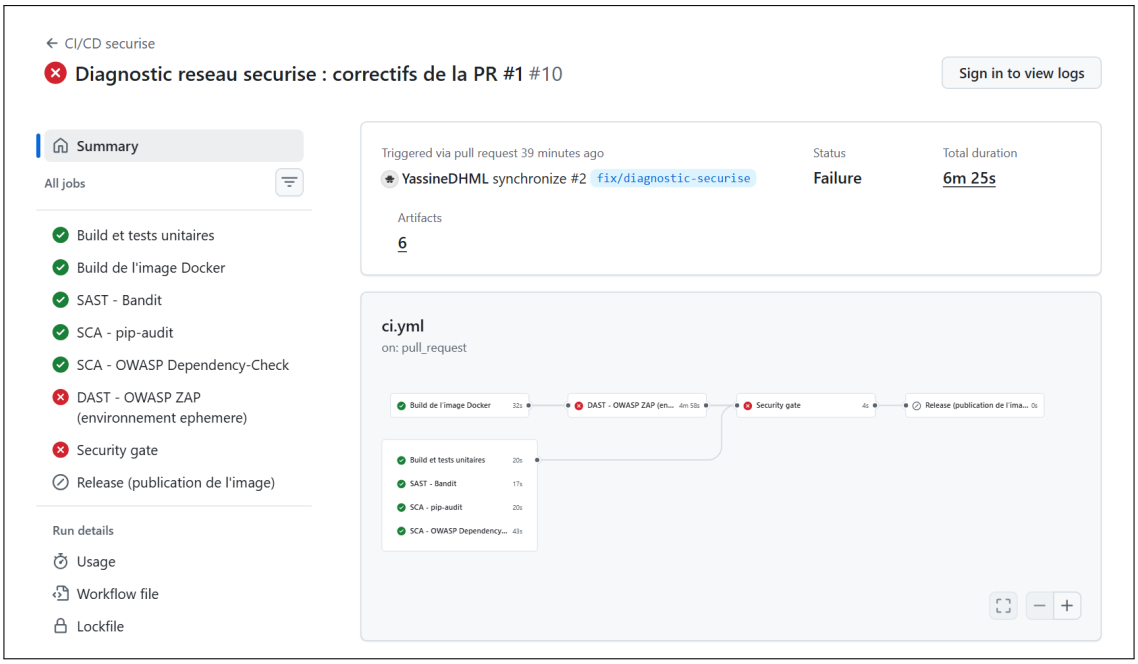


FIGURE 8 – Exécution n°10 de la PR corrigée : seul le job DAST (et donc la porte de sécurité) est en échec.

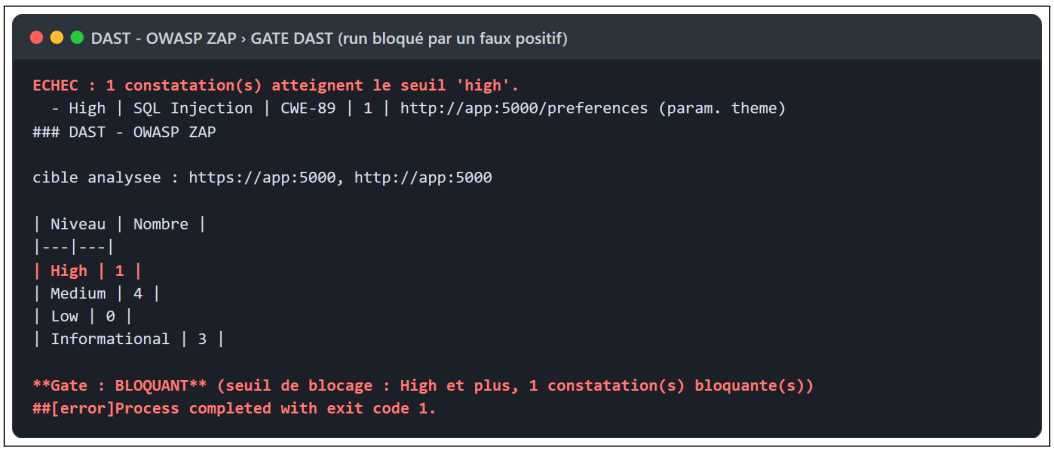
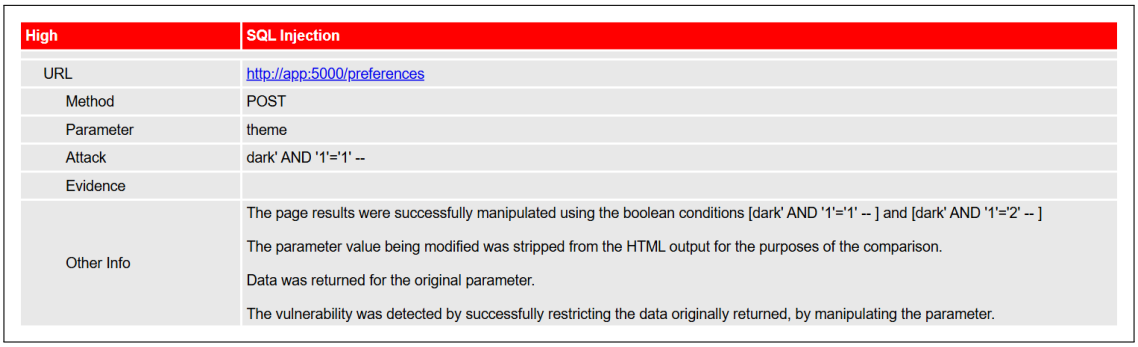


FIGURE 9 – Le gate DAST signale une injection SQL (High) sur `POST /preferences`, paramètre `theme`.

Le gate a fait son travail (une alerte High bloque). Reste à décider si l'alerte est fondée : c'est le **triage**, en trois temps.

1. **Lire l'alerte** (Figure 10). ZAP a utilisé sa règle de détection *booléenne* : il compare les pages obtenues avec `dark' AND '1'='1' --` et `dark' AND '1'='2' --`. Il n'y a ni message d'erreur SQL ni donnée extraite : la seule preuve est une différence entre deux pages, avec une confiance *Medium*.
2. **Lire le code.** Dans `/preferences`, `theme` est comparé à la liste blanche `{"light", "dark"}` puis rangé dans le cookie de session. **Aucune requête SQL** : la route n'utilise même pas la base de données.
3. **Expliquer le mécanisme** (hypothèse la plus probable). La page affichée dépend de la valeur stockée dans la **session**, que les requêtes du scanner modifient en permanence : le scan actif utilise plusieurs fils d'exécution qui partagent la même session. Deux réponses comparées par

ZAP peuvent donc différer parce que l'état a changé entre-temps, pas parce que la saisie a modifié une requête SQL. Sans requête SQL, il n'y a de toute façon pas d'injection SQL possible.



High	SQL Injection
URL	http://app:5000/preferences
Method	POST
Parameter	theme
Attack	dark' AND '1'='1' --
Evidence	
Other Info	The page results were successfully manipulated using the boolean conditions [dark' AND '1'='1' --] and [dark' AND '1'='2' --] The parameter value being modified was stripped from the HTML output for the purposes of the comparison. Data was returned for the original parameter. The vulnerability was detected by successfully restricting the data originally returned, by manipulating the parameter.

FIGURE 10 – Détail de l'alerte dans le rapport ZAP : seule une différence de pages, aucune preuve d'exécution SQL.

Conclusion : faux positif. Le traitement est un **filtre étroit, documenté et gardé par un test** :

— un job `alertFilter` de `zap/automation.yaml` requalifie cette alerte en *False Positive* pour **une seule combinaison** : règle 40018, URL, paramètre `theme`, méthode POST. La règle reste active partout ailleurs. Un essai préalable sur une autre alerte a confirmé que le filtre n'écarte que l'instance visée (5 instances ramenées à 4) ;

— le test `test_preferences_never_touch_the_database` **verrouille l'hypothèse** : si une évolution du code faisait utiliser SQL à cette route, le test échouerait et le filtre devrait être retiré ;

— les preuves du run bloqué sont conservées dans `reports/pr2-faux-positif/`.

Ce qu'on ne fait **pas** : relancer le job jusqu'à ce qu'il passe (cela masquerait aussi une vraie vulnérabilité intermittente), baisser le seuil global ou désactiver la règle d'injection SQL. Le run suivant, avec le filtre, est repassé au vert (Figure 11).

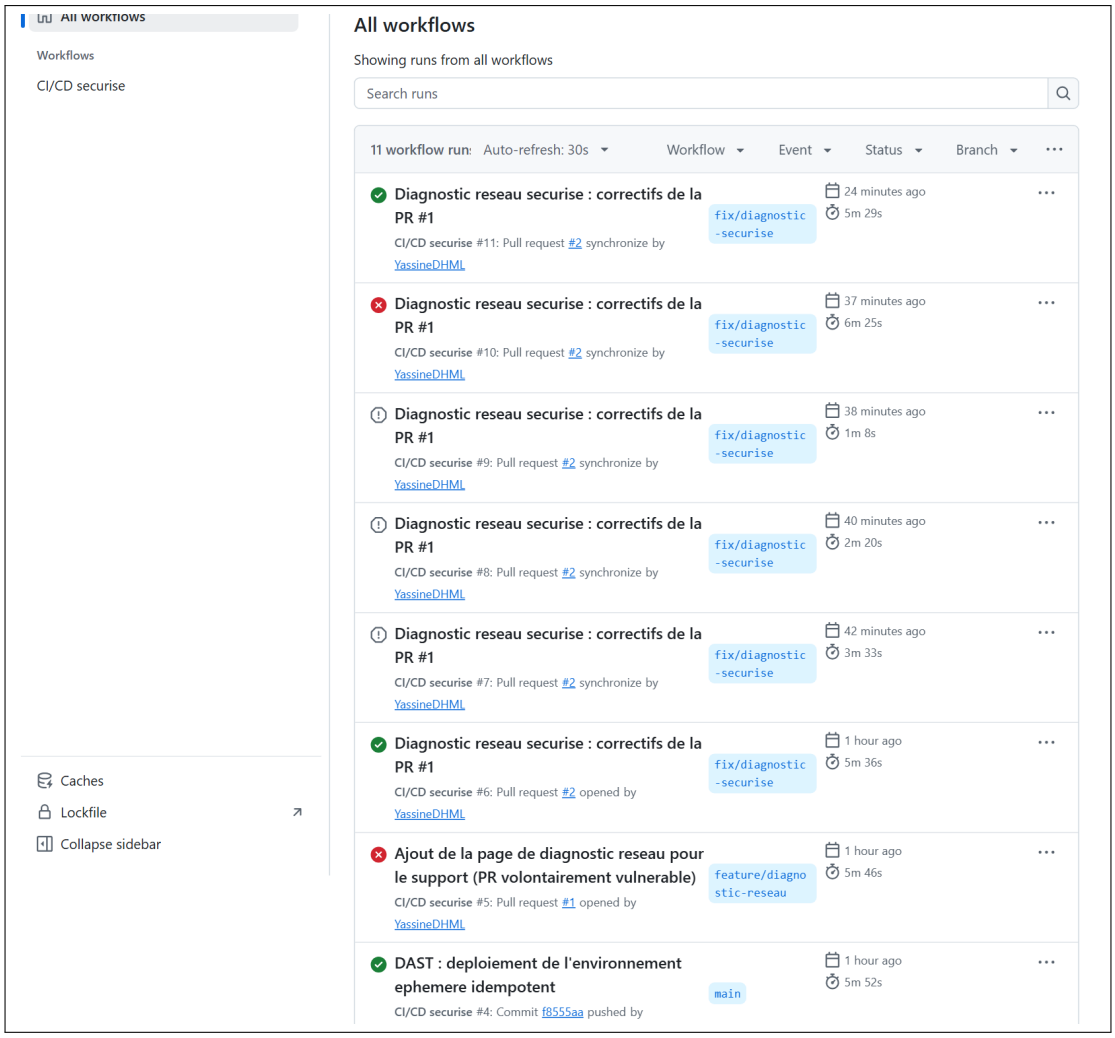


FIGURE 11 – Liste des exécutions du pipeline : #5 en échec (PR vulnérable), #6 vert, #7 à #9 annulées par un *push* plus récent, #10 bloquée par le faux positif, #11 verte.

6 Partie IV – Porte de sécurité, protection de la branche et publication

6.1 Le job `security-gate`

Listing 4 – Porte de sécurité : agrégat de tous les contrôles

```
security-gate:
  name: Security gate
  needs: [build-test, build-image, sast, sca-pip-audit, sca-dependency-check,
    dast]
  if: always()
  steps:
    - name: Bilan des controles
      env:
        NEEDS: ${{ toJSON(needs) }}
      run: |
        ...
        if echo "$NEEDS" | jq -e 'to_entries | any(.value.result != "success")'
    > /dev/null; then
          echo "::error::Au moins un controle n'est pas vert : version non
    securisee, publication bloquee."
```

```
          exit 1
        fi
```

Deux détails importants :

— `if: always()`. Par défaut, un job dont une dépendance a échoué est *ignoré*. Or GitHub considère un contrôle obligatoire ignoré comme **satisfait** : sans `if: always()`, un échec du SAST laisserait passer la fusion. Le job doit donc s'exécuter dans tous les cas et échouer lui-même.

— `result != "success"`. Le gate échoue non seulement si un contrôle est en échec, mais aussi s'il est *ignoré* ou *annulé* : seul un contrôle explicitement vert compte.

6.2 Protection de la branche `main`

Un pipeline qui échoue ne protège rien si l'on peut fusionner quand même. La branche `main` est donc protégée (règle créée avec l'API REST de GitHub, Figure 12) :

Réglage	Valeur	Effet
Pull request obligatoire	oui	Aucun *push* direct sur `main` (0 approbation requise : dépôt personnel)
Contrôle exigé	**Security gate**	La fusion est impossible tant que ce job n'est pas vert
Branche à jour avant fusion	oui	Le contrôle a bien porté sur le code qui sera fusionné
Inclure les administrateurs	**oui**	Personne, pas même le propriétaire, ne contourne la règle
Force push et suppression	interdits	L'historique de `main` ne peut pas être réécrit

Protection de la branche main (API GitHub)

```
$ gh api repos/YassineDHML/tp2-devsecops-cicd-pipeline/branches/main/protection --jq '{...}'
{
    "allow_deletions": false,
    "allow_force_pushes": false,
    "enforce_admins": true,
    "required_pull_request_reviews": {
        "required_approving_review_count": 0
    },
    "required_status_checks": {
        "contexts": [
            "Security gate"
        ],
        "strict": true
    }
}
```

FIGURE 12 – Règle de protection de `main`, telle que renvoyée par l'API GitHub.

La page des branches affiche l'icône de protection à côté de `main`, avec le résultat des contrôles de chaque branche (Figure 13).

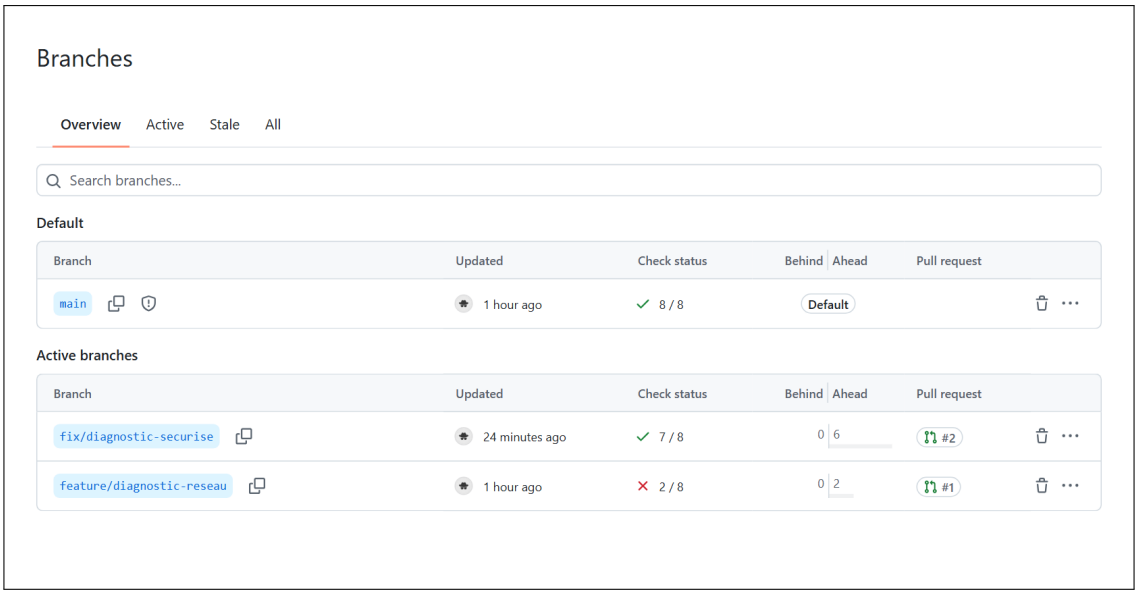


FIGURE 13 – Page des branches : `main` protégée (bouclier), 8/8 contrôles réussis.

6.3 Le job `release` : ne jamais publier une version non sécurisée

Listing 5 – Conditions de la publication

```
release:
  needs: security-gate
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  permissions:
    contents: read
    packages: write
```

Le job télécharge l'image **testée par le DAST** et la publie sur GitHub Container Registry avec deux étiquettes : le SHA du commit et `latest`. Il ne s'exécute que sur un *push* sur `main` et si `security-gate` est vert : **aucun chemin ne publie une image non validée**. Le premier run complet sur `main` le démontre (Figures 14 et 15).

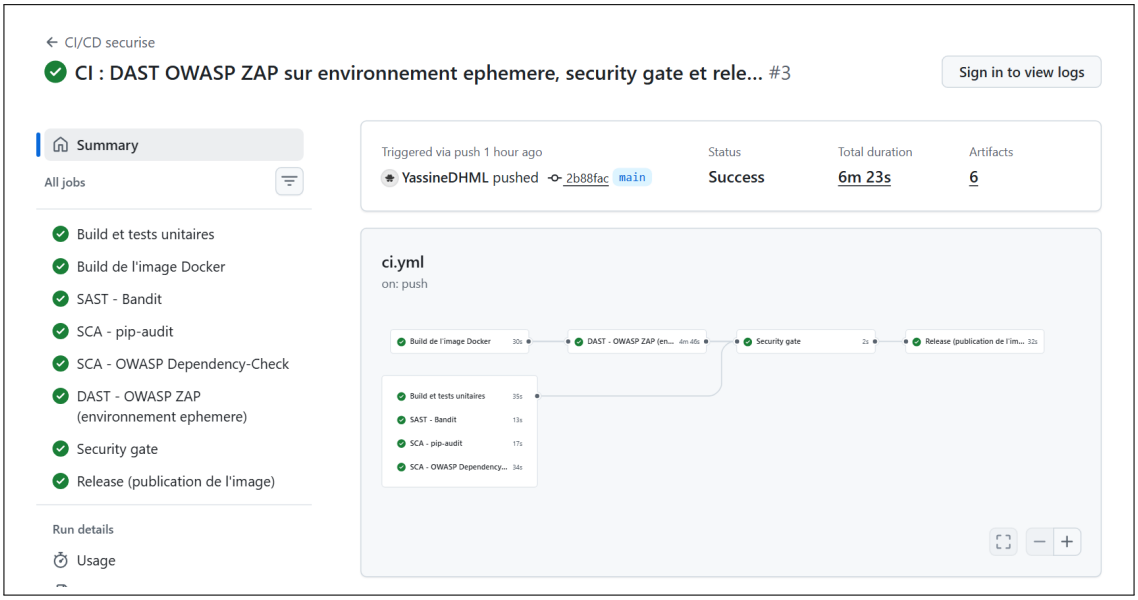


FIGURE 14 – Premier run complet sur `main` : les huit jobs sont verts, dont la publication.

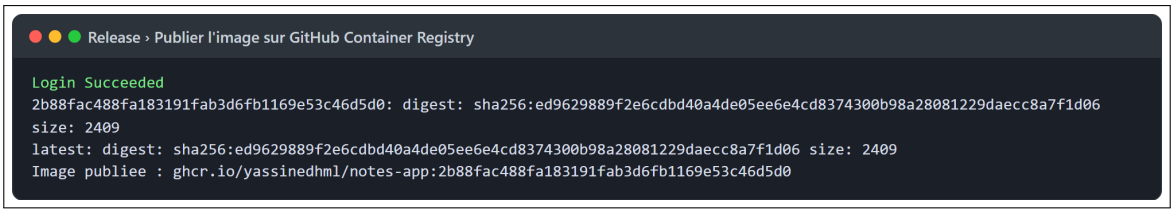


FIGURE 15 – Journal de la publication : les deux étiquettes pointent vers le même *digest*.

7 Partie V – Validation : pull request vulnérable, puis corrigée

7.1 Scénario

Un développeur ajoute une page de diagnostic réseau pour le support : `/diagnostic?host=<nom>` affiche la résolution DNS d'un nom d'hôte. Deux pull requests proposent cette fonctionnalité à partir de `main`, branche protégée (Figure 16). Elles ont été ouvertes en même temps et exécutées en parallèle.

Pull request	Branche	Contenu	Exécution
n° 1	`feature/diagnostic-reseau`	Version **volontairement vulnérable** et dépendance ancienne	#5
n° 2	`fix/diagnostic-securise`	Même fonctionnalité, **corrigée**	#6 puis #11

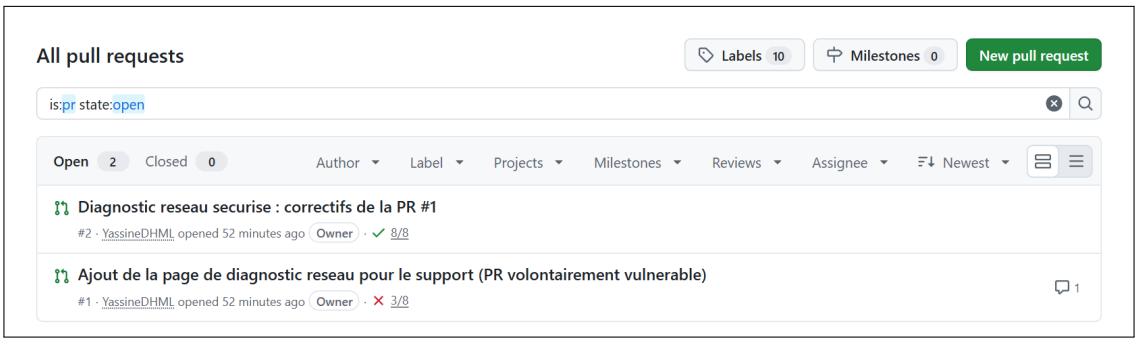


FIGURE 16 – Liste des pull requests : la n° 2 est verte (8/8), la n° 1 est en échec (3/8).

7.2 Pull request n° 1 : la version vulnérable est bloquée

Listing 6 – Code de la route `/diagnostic` proposée par la PR n° 1

```
@app.route("/diagnostic")
def diagnostic():
    host = request.args.get("host", "localhost")
    result = subprocess.run("getent hosts " + host, shell=True, capture_output=
    True, text=True, timeout=10)
    return "<h1>Resolution de " + host + "</h1><pre>" + (result.stdout or "aucun
     resultat") + "</pre>"
```

Un lien « Diagnostic » est ajouté dans la barre de navigation, un test vérifie le cas nominal, et `requirements.txt` épingle `certifi==2023.5.7`. Quatre défauts sont introduits :

#	Défaut	Catégorie
1	Route **publique** : pas de `@login_required`	Contrôle d'accès défaillant (OWASP A01)
2	`shell=True` avec `host` concaténé dans la commande	**Injection de commande** (CWE-78, OWASP A03)
3	`host` recopié dans une chaîne HTML renvoyée telle quelle	**XSS réfléchi** (CWE-79, OWASP A03)
4	`certifi==2023.5.7`	Composant vulnérable (OWASP A06) : CVE-2023-37920 (CVSS 9.8) et CVE-2024-39689 (CVSS 7.5)

Remarque : `requests` 2.34.2 exige `certifi>=2023.5.7` ; c'est la plus ancienne version que `pip` accepte, et elle est vulnérable. Un premier essai avec `certifi==2022.12.7` avait échoué dès la résolution des dépendances.

Résultat du pipeline

Job	Résultat	Durée
Build et tests unitaires	réussi (67 tests)	25 s
Build de l'image Docker	réussi	28 s
SAST – Bandit	**échec**	15 s
SCA – pip-audit	**échec**	18 s
SCA – OWASP Dependency-Check	**échec**	34 s
DAST – OWASP ZAP	**échec**	5 min 07 s
Security gate	**échec**	4 s
Release	ignorée	–

Table 6 – Pipeline de la PR n° 1 (exécution #5)

Tous les contrôles se sont exécutés, puisqu'ils sont indépendants : **un seul run donne la liste complète des problèmes**. La publication est ignorée et la protection de `main` bloque la fusion (Figures 17, 18 et 19).



FIGURE 17 – Pull request n° 1 : le dernier commit est marqué en échec, et le commentaire récapitule les contrôles.

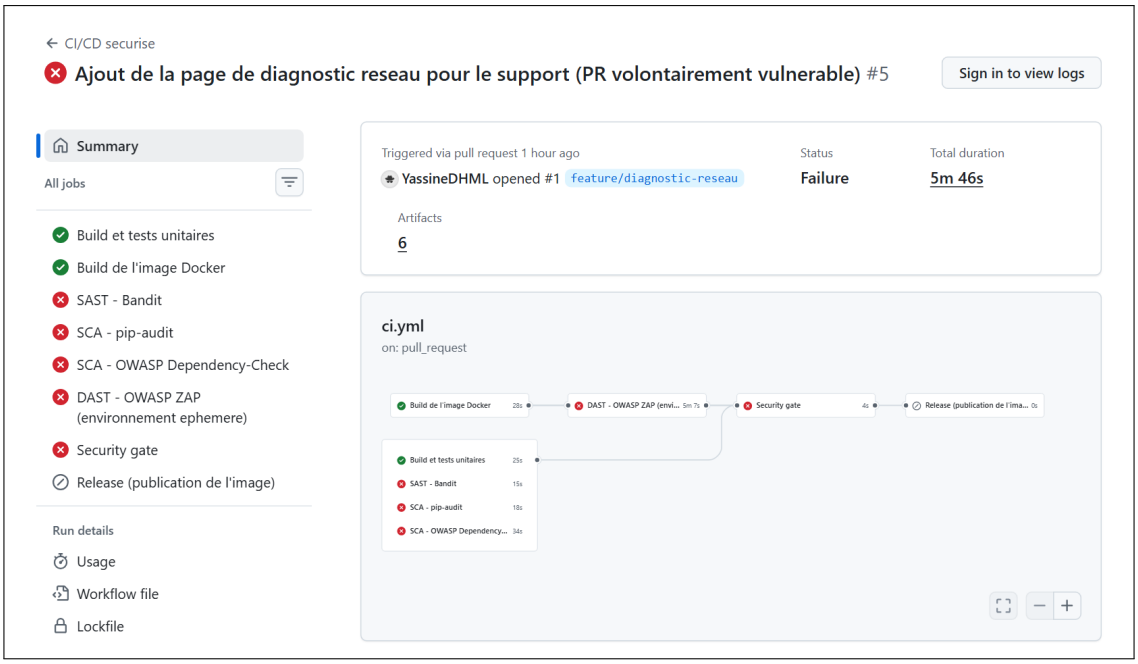


FIGURE 18 – Exécution #5 : build et tests réussis, SAST, SCA, DAST et porte de sécurité en échec, release ignorée.

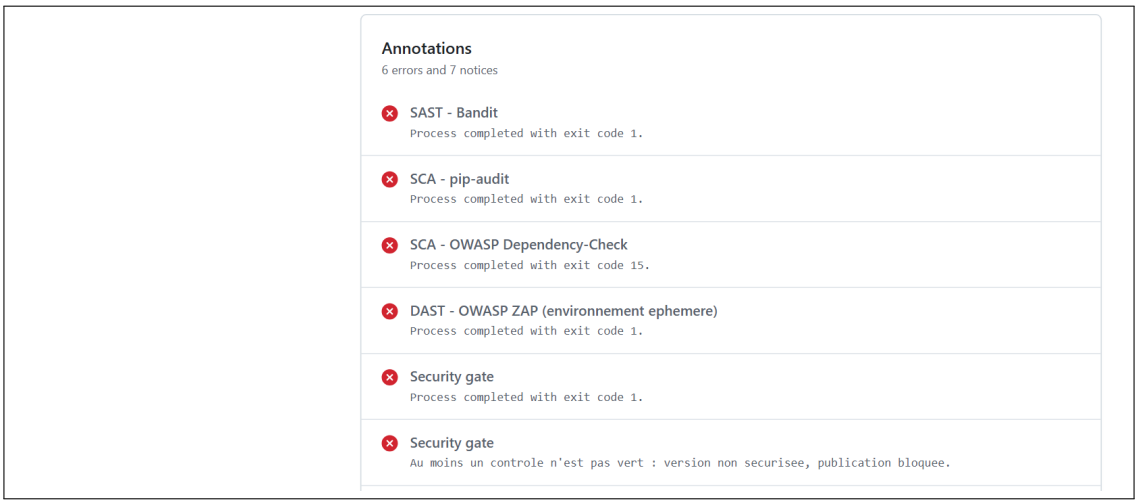


FIGURE 19 – Annotations de l'exécution #5 : code de sortie 1 pour Bandit, pip-audit, ZAP et la porte, code 15 pour Dependency-Check.

SAST : Bandit détecte l'injection de commande

Le gate signale la règle `B602` (sévérité High, confiance High) à la ligne 287 (Figures 20 et 21). Les deux alertes *Low* sont celles de l'application saine ; seule la High bloque.



FIGURE 20 – Journal du gate SAST : `B602` (High) et code de sortie 1.

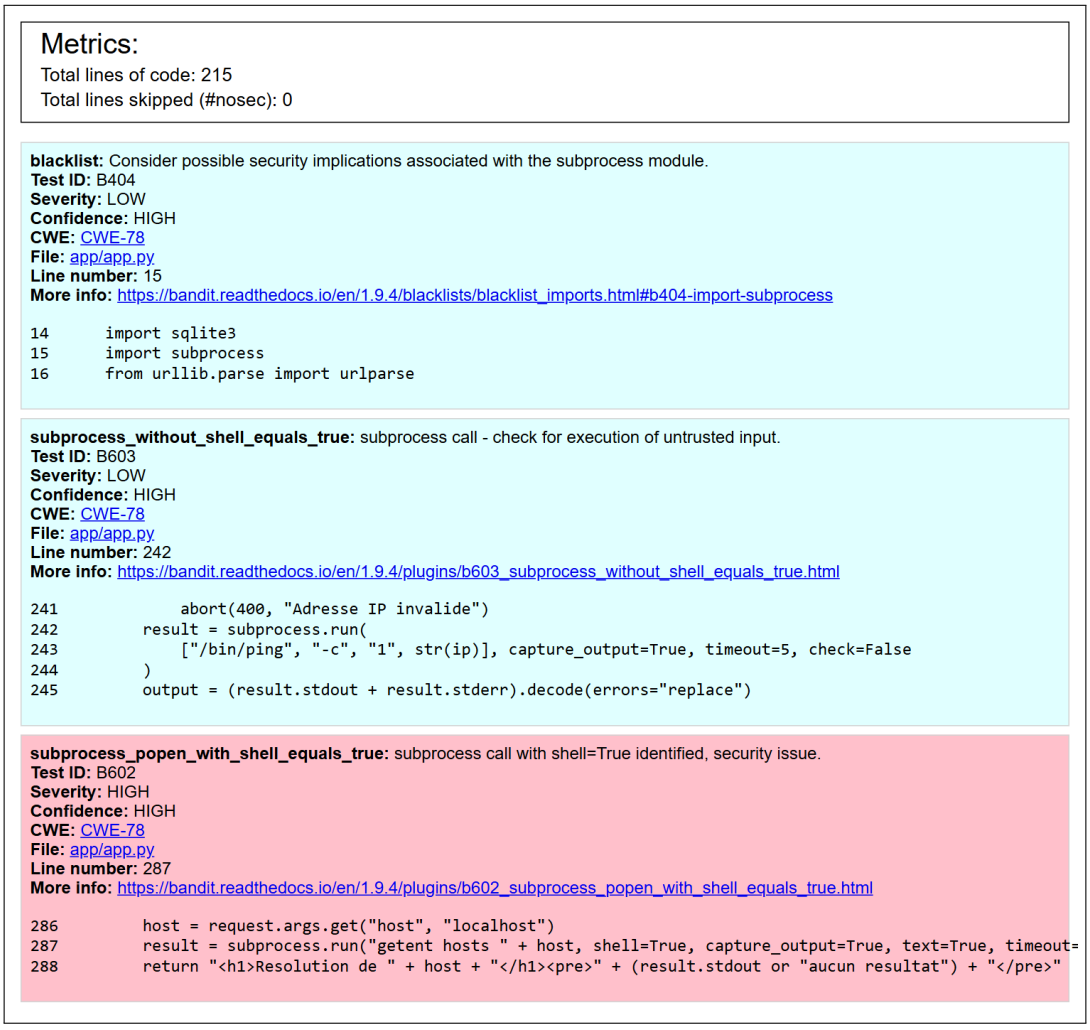


FIGURE 21 – Rapport HTML de Bandit sur la PR n° 1 : `B602` en rouge.

SCA : pip-audit et Dependency-Check détectent `certifi`

pip-audit signale 3 avis (6 lignes, chacun répété une fois par version corrigée), Dependency-Check 2 CVE de score élevé (Figures 22, 23 et 24).

SCA - pip-audit › GATE SCA (1/2) : échec si une dépendance a une vulnérabilité connue

```
pip-audit -r requirements.txt --no-deps --disable-pip --desc off
Found 6 known vulnerabilities in 1 package
Name    Version  ID              Fix Versions
------- -------- --------------- ------------
certifi 2023.5.7 PYSEC-2023-135  2023.7.22
certifi 2023.5.7 PYSEC-2023-135  2023.7.22
certifi 2023.5.7 PYSEC-2024-230  2024.7.4
certifi 2023.5.7 PYSEC-2024-230  2024.7.4
certifi 2023.5.7 PYSEC-2026-4320 2026.10.18
certifi 2023.5.7 PYSEC-2026-4320 2026.10.19
##[error]Process completed with exit code 1.
```

FIGURE 22 – Journal du gate pip-audit : `certifi` 2023.5.7 est vulnérable.

Project: TP2 Notes App

Scan Information (show all):
- *dependency-check version*: 13.0.0
- *Report Generated On*: Thu, 29 Oct 2026 00:33:13 GMT
- *Dependencies Scanned*: 14 (14 unique)
- *Vulnerable Dependencies*: 1
- *Vulnerabilities Found*: 2
- *Vulnerabilities Suppressed*: 0
- ...

Summary

Summary of Vulnerable Dependencies (click to show all)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
certifi:2023.5.7	cpe:2.3:a:certifi:certifi:2023.5.7:*:*:*:*:*:*:*	pkg:pypi/certifi@2023.5.7	CRITICAL	2	Highest	3

Dependencies (vulnerable)

certifi:2023.5.7

File Path: /src/requirements.txt:certifi/2023.5.7

Evidence

Identifiers

- pkg:pypi/certifi@2023.5.7 (*Confidence*:Highest)
- cpe:2.3:a:certifi:certifi:2023.5.7:*:*:*:*:*:*:* (*Confidence*:Highest) suppress

Published Vulnerabilities

CVE-2023-37920 suppress

Certifi is a curated collection of Root Certificates for validating the trustworthiness of SSL certificates while verifying the identity of TLS hosts. Certifi prior to version 2023.07.22 recognizes "e-Tugra" root certificates. e-Tugra's root certificates were subject to an investigation prompted by reporting of security issues in their systems. Certifi 2023.07.22 removes root certificates from "e-Tugra" from the root store.

CWE-345 Insufficient Verification of Data Authenticity

CVSSv3:
- Base Score: CRITICAL (9.8)
- Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H/E:P/RC:R/MAV:A

References:

FIGURE 23 – Rapport Dependency-Check : `certifi:2023.5.7`, CVE-2023-37920 (CVSS 9.8, Critical) et CVE-2024-39689 (CVSS 7.5, High).

SCA - OWASP Dependency-Check › GATE SCA (2/2) : échec si CVSS ≥ 7

```
[INFO] Analysis Complete (6 seconds)
[ERROR]

One or more dependencies were identified with vulnerabilities that have a CVSS score greater than or equal to '7.0':

requirements.txt (pkg:pypi/certifi@2023.5.7, cpe:2.3:a:certifi:certifi:2023.5.7:*:*:*:*:*:*:*): CVE-2023-37920(9.8), CVE-
2024-39689(7.5)

See the dependency-check report for more details.

##[error]Process completed with exit code 15.
```

FIGURE 24 – Journal du gate Dependency-Check : seuil CVSS dépassé, code de sortie 15.

DAST : ZAP prouve l'exploitation

ZAP relève **2 alertes High** sur `/diagnostic` (Figures 25, 26 et 27) :

ZAP by Checkmarx **TP2 DevSecOps - scan DAST de la Notes App**

Sites: https://app:5000 http://app:5000

Generated on Thu, 29 Oct 2026 00:38:10

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	2
Medium	4
Low	0
Informational	3
False Positives:	0

Alerts

Name	Risk Level	Number of Instances
Cross Site Scripting (Reflected)	High	1
Remote OS Command Injection	High	1
Absence of Anti-CSRF Tokens	Medium	Systemic
CSP: Failure to Define Directive with No Fallback	Medium	Systemic
CSP: style-src unsafe-inline	Medium	Systemic
HTTP Only Site	Medium	1
Authentication Request Identified	Informational	1
Session Management Response Identified	Informational	2
User Agent Fuzzer	Informational	Systemic

FIGURE 25 – Rapport ZAP sur la PR n° 1 : 2 alertes High, en plus des 4 Medium de l'application saine.

High	Remote OS Command Injection
URL	http://app:5000/diagnostic?host=localhost%26cat+%2Fetc%2Fpasswd%26
Method	GET
Parameter	host
Attack	localhost&cat /etc/passwd&
Evidence	root:x:0:0
Other Info	The scan rule was able to retrieve the content of a file or command by sending [localhost&cat /etc/passwd&] to the operating system running this application.

FIGURE 26 – *Remote OS Command Injection* : ZAP a envoyé `localhost&cat /etc/passwd&` et retrouvé `root:x:0:0` dans la réponse.

High	Cross Site Scripting (Reflected)
URL	http://app:5000/diagnostic?host=%3C%2Fh1%3E%3CscrIpt%3Ealert%281%29%3B%3C%2FscRipt%3E%3Ch1%3E
Method	GET
Parameter	host
Attack	<scrIpt>alert(1);</scRipt>
Evidence	<scrIpt>alert(1);</scRipt>
Other Info	

FIGURE 27 – *Cross Site Scripting (Reflected)* : la charge utile est recopiée telle quelle dans la page.

DAST - OWASP ZAP › GATE DAST : échec si une alerte de risque High est détectée

```
bash scripts/dast.sh gate

### DAST - OWASP ZAP
ECHEC : 2 constatation(s) atteignent le seuil 'high'.
  - High | Cross Site Scripting (Reflected) | CWE-79 | 1 | http://app:5000/diagnostic?
host=%3C%2Fh1%3E%3CscrIpt%3Ealert%281%29%3B%3C%2Fs...
  - High | Remote OS Command Injection | CWE-78 | 1 | http://app:5000/diagnostic?host=localhost%26cat+%2Fetc%2Fpasswd%26
(param. host)

cible analysee : https://app:5000, http://app:5000

| Niveau | Nombre |
|---|---|
| High | 2 |
| Medium | 4 |
| Low | 0 |
| Informational | 3 |

**Gate : BLOQUANT** (seuil de blocage : High et plus, 2 constatation(s) bloquante(s))
##[error]Process completed with exit code 1.
```

FIGURE 28 – Journal du gate DAST sur la PR n° 1 : deux alertes High, code de sortie 1.

Le contenu de `/etc/passwd` dans la réponse est la **preuve** que la commande a été exécutée par le serveur. La même attaque, rejouée à la main sur l'image de la PR n° 1, donne la Figure 29 ; le HTML injecté est interprété par le navigateur (Figure 30). La politique CSP de l'application empêche l'exécution d'un script *inline*, mais pas l'injection de balises.



FIGURE 29 – Exploitation manuelle de la PR n° 1 : le serveur renvoie `/etc/passwd`.



FIGURE 30 – Injection de HTML dans la réponse : le titre rouge n'existe pas dans le code de la page.

La porte de sécurité bloque la fusion

Listing 7 – Extrait de la sortie de la porte de sécurité sur la PR n° 1

```
| Controle | Resultat |
| build-test | success |
| build-image | success |
| sast | failure |
| sca-pip-audit | failure |
| sca-dependency-check | failure |
| dast | failure |
##[error]Au moins un controle n'est pas vert : version non securisee,
    publication bloquee.
```



FIGURE 31 – Journal de la porte de sécurité sur la PR n° 1.



FIGURE 32 – État de fusion renvoyé par l'API : `BLOCKED` pour la PR n° 1, `CLEAN` pour la PR n° 2.

Ce que chaque contrôle a vu

Défaut de la PR n° 1	Tests	Bandit	pip-audit	Dep.-Check	ZAP
1. Route publique	non vu	non vu	–	–	non signalé en tant que tel
2. Injection de commande	non vue	**détectée** (B602)	–	–	**détectée** (High)
3. XSS réfléchi	non vu	**non vu**	–	–	**détecté** (High)
4. `certifi` vulnérable	non vue	–	**détectée**	**détectée**	–

TABLE 7 – Matrice défauts × contrôles

1. **Les tests unitaires sont verts.** Ils vérifient que la fonctionnalité marche, pas qu'elle est sûre : seuls les contrôles de sécurité arrêtent cette PR.
2. **SAST et DAST se complètent.** L'injection de commande est vue par les deux : Bandit repère le motif `shell=True`, ZAP prouve l'exécution. Le XSS n'est vu **que par le DAST** : `"<h1>" + host + "</h1>"` ne contient aucun motif que Bandit sache reconnaître.
3. **Les deux outils SCA concordent** sur le même composant, avec des vocabulaires différents (avis PyPA d'un côté, CVE et score CVSS de l'autre).

4. **Aucun outil ne signale l'absence d'authentification en tant que telle** : le contrôle d'accès relève de la logique métier. Cette règle doit être verrouillée par un **test unitaire**, ce que fait la PR n° 2.

7.3 Pull request n° 2 : la version corrigée passe

Listing 8 – Code corrigé de la route `/diagnostic` (PR n° 2)

```
HOSTNAME_RE = re.compile(
    r"(?=.{1,253}\Z)(?!-)[A-Za-z0-9-]{1,63}(?<!-)(\.(?!-)[A-Za-z0
    -9-]{1,63}(?<!-))*"
)

@app.route("/diagnostic")
@login_required
def diagnostic():
    host = request.args.get("host", "localhost")
    if not HOSTNAME_RE.fullmatch(host):
        abort(400, "Nom d'hote invalide")
    try:
        infos = socket.getaddrinfo(host, None, proto=socket.IPPROTO_TCP)
    except socket.gaierror:
        output = "Aucune adresse trouvee"
    else:
        output = "\n".join(sorted({info[4][0] for info in infos}))
    return render_template("tool.html", title="Resolution de " + host, output=
    output)
```

Défaut de la PR n° 1	**Correction**
Route publique	`@login_required` : redirection vers `/login` sans session ; lien affiché aux seuls utilisateurs connectés
Injection de commande	**Plus aucun sous-processus** : résolution avec `socket.getaddrinfo`. Le danger est supprimé à la source plutôt que filtré
Entrée non contrôlée	Nom d'hôte validé par **liste blanche** (RFC 1123, `fullmatch`), erreur 400 sinon
XSS réfléchi	Réponse rendue par un **template Jinja2** (auto-échappement) au lieu d'une chaîne HTML concaténée
`certifi` ancien	Pas d'épinglage : `main` garde `certifi==2026.10.23`

Les tests ajoutés verrouillent ces propriétés : authentification obligatoire, 9 noms d'hôte dangereux rejetés (`;`, `$( )`, `&&`, `|`, `-c1`, balise `<script>`, saut de ligne, 300 caractères), aucun appel à `subprocess`, hôte inconnu.

Résultat du pipeline

Job	Résultat	Durée
Build et tests unitaires	réussi (78 tests)	26 s
Build de l'image Docker	réussi	28 s
SAST – Bandit	réussi (0 High, 2 Low acceptées)	17 s
SCA – pip-audit	réussi (0 vulnérabilité)	17 s
SCA – OWASP Dependency-Check	réussi (0 CVE)	38 s
DAST – OWASP ZAP	réussi (**0 High**, 4 Medium)	4 min 57 s
Security gate	**réussi**	3 s
Release	ignorée sur une PR	–

TABLE 8 – Pipeline de la PR n° 2 (exécution #6, commit `c93889e`)

L'état de la PR est `CLEAN` (Figure 32) : toutes les règles de la protection de `main` sont satisfaites et la fusion est autorisée. L'historique de la PR (Figure 33) montre aussi l'épisode du faux positif de la §5.6 : les croix intermédiaires sont des exécutions annulées par une poussée plus récente, puis le run bloqué, avant le retour au vert.



FIGURE 33 – Pull request n° 2 : 6 commits, premier et dernier commits validés.

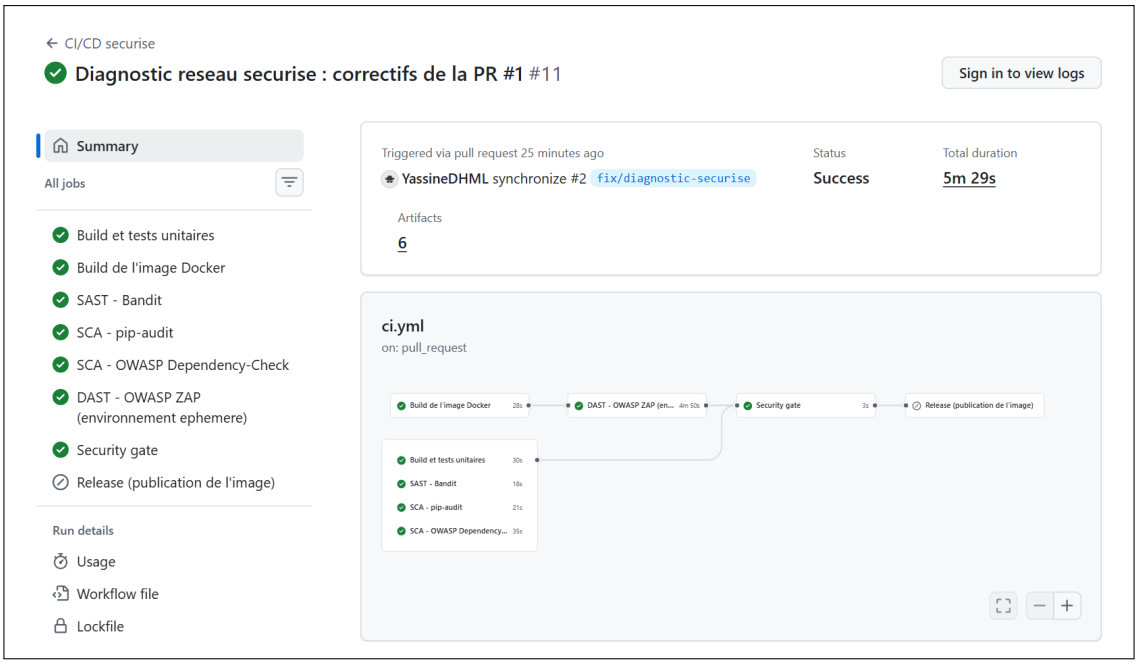


FIGURE 34 – Exécution #11 de la PR n° 2 : tous les contrôles sont verts, release ignorée (événement `pull_request`).

Les trois scanners n'ont plus rien à signaler (Figures 35 et 36, et Figure 7 pour ZAP) :



FIGURE 35 – Rapport Bandit sur la PR n° 2 : plus de `B602`, seules restent les deux alertes Low acceptées.



FIGURE 36 – Rapport Dependency-Check sur la PR n° 2 : 14 dépendances analysées, aucune vulnérable.

Comportement de l'application corrigée

Sans session, la route redirige vers la page de connexion (Figure 37). Connecté, un nom d'hôte invalide est refusé (HTTP 400, Figure 38) et un nom valide renvoie ses adresses (Figure 39).



FIGURE 37 – Version corrigée : accès sans session, redirection vers la page de connexion.

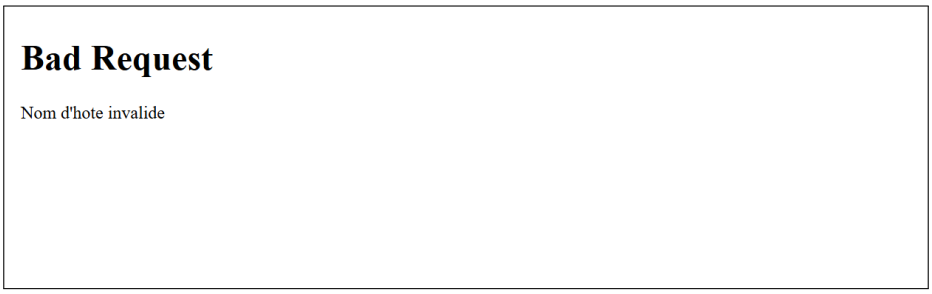


FIGURE 38 – Version corrigée : la tentative `localhost; id` est refusée (HTTP 400).

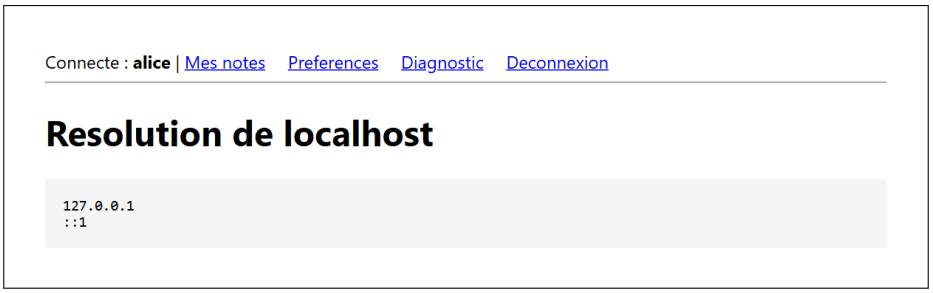


FIGURE 39 – Version corrigée : un nom d'hôte valide renvoie ses adresses.

7.4 Publication de l'image

Le job `release` est **ignoré sur une pull request** (`github.event_name == 'push'` est faux) : c'est ce que l'on observe sur les deux PR. Il s'exécutera après la fusion, sur le *push* résultant sur `main`, et seulement si la porte de sécurité est verte, comme l'a démontré le run de la Figure 14.

7.5 Synthèse comparée

Contrôle	PR n° 1 (vulnérable)	PR n° 2 (corrigée)
Tests unitaires	67 réussis	78 réussis
Bandit	1 High, 2 Low : **bloque**	0 High, 2 Low : passe
pip-audit	3 avis sur `certifi` : **bloque**	0 : passe
Dependency-Check	1 Critical (9.8), 1 High (7.5) : **bloque**	0 CVE : passe
ZAP	2 High, 4 Medium : **bloque**	0 High, 4 Medium : passe
Porte de sécurité	**échec**, état `BLOCKED`	réussie, état `CLEAN`
Publication	ignorée	ignorée (PR), exécutée après la fusion

TABLE 9 – Bilan comparé des deux pull requests

8 Conclusion

Ce TP a transformé les contrôles ponctuels du TP1 en un **garde-fou permanent**. Chaque modification traverse un pipeline qui compile, teste, analyse le code (Bandit), les dépendances (pip-audit, Dependency-Check) et l'application en cours d'exécution (OWASP ZAP sur un environnement éphémère). Un job unique, `Security gate`, agrège les résultats ; la branche `main` est protégée, y compris pour l'administrateur, et l'image n'est publiée que si tout est vert. La validation l'a démontré : la pull request vulnérable, dont les tests unitaires passaient, a été arrêtée par quatre contrôles distincts, alors que la version corrigée est passée sans alerte bloquante.

Le TP met en évidence plusieurs points :

— **Des tests fonctionnels verts ne disent rien de la sécurité.**

— **SAST et DAST se complètent** : le XSS n'est vu que par le DAST ; l'injection de commande l'est par les deux.

— **La SCA doit être continue** : `certifi` est devenu vulnérable sans qu'une ligne de code change, et deux outils SCA ne voient pas les mêmes choses.

— **Un garde-fou doit échouer fermé** : rapport manquant, contrôle ignoré, scan vide, tous ces cas font échouer le pipeline. Un contrôle obligatoire *ignoré* passe pour réussi dans la protection de branche, d'où le `if: always()` de la porte de sécurité.

— **Un DAST produit des faux positifs** : il faut un processus de triage, pas un « relancer jusqu'à ce que ça passe » ni une baisse du seuil.

Limites. La couverture du DAST dépend de ce que ZAP connaît (les routes non liées doivent être déclarées) ; aucun scanner n'est exhaustif (Dependency-Check n'a pas vu l'avis `certifi`, Bandit ne voit pas le XSS) ; le pipeline lui-même est une cible : une PR s'exécute avec sa propre version de `ci.yml`, d'où l'intérêt d'un `CODEOWNERS` imposant une revue sur `.github/`.

Pistes d'amélioration. Corriger les alertes Medium du DAST (protection CSRF avec Flask-WTF, CSP complète, HTTPS), Dependabot pour les dépendances et les actions, analyse de l'image avec Trivy, recherche de secrets avec gitleaks, SBOM et signature de l'image (cosign), approbation manuelle avant la publication, scan DAST complet nocturne.

Annexe A – Fichier `.github/workflows/ci.yml`

Listing 9 – Pipeline complet (fichier `.github/workflows/ci.yml`)

```
# Pipeline CI/CD du TP2 : build, tests unitaires, SAST, SCA, DAST, puis publication.
#
#   build-test ----------------------------\
#   build-image --+--> dast ----------------\
#   sast ---------------------------------- +--> security-gate --> release (main
    uniquement)
#   sca-pip-audit -------------------------/
#   sca-dependency-check ------------------/
#
# Chaque controle bloque le pipeline des qu'une vulnerabilite de severite haute ou
# critique est detectee. Le job "security-gate" (seul controle exige par la
# protection de la branche main) agrege les resultats, et "release" ne s'execute
# que s'il est vert : une version non securisee n'est jamais publiee.
name: CI/CD securise

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  schedule:
    # Chaque lundi : de nouvelles CVE sont publiees meme sans nouveau commit
    - cron: "17 5 * * 1"
  workflow_dispatch:

# Moindre privilege : le jeton du workflow ne peut que lire le depot.
permissions:
  contents: read

concurrency:
  group: ci-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}

env:
  PYTHON_VERSION: "3.13"
  # Seuils de blocage (voir RAPPORT.md, section 4.1 "Politique de blocage")
  BANDIT_FAIL_LEVEL: high      # severite Bandit minimale qui bloque (low | medium | high
    )
  DC_FAIL_ON_CVSS: "7"         # score CVSS minimal qui bloque (7.0 = High, 9.0 =
    Critical)

jobs:
  # ------------------------------------------------------------------ phase 1
  build-test:
    name: Build et tests unitaires
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
      - uses: actions/setup-python@6d4fd22b20465ccc0f7354476588e8df7c9ca0e0 # v7.1.0
        with:
          python-version: ${{ env.PYTHON_VERSION }}
```

```
          cache: pip
          cache-dependency-path: requirements*.txt
      - name: Installer les dependances
        run: python -m pip install -r requirements-dev.txt
      - name: Compiler le code (verification syntaxique)
        run: python -m compileall -q app tests scripts
      - name: Tests unitaires et couverture (seuil 90 %)
        run: |
          mkdir -p ci-reports
          pytest --junitxml=ci-reports/junit.xml \
                 --cov=app --cov-report=term-missing \
                 --cov-report=xml:ci-reports/coverage.xml \
                 --cov-fail-under=90
      - uses: actions/upload-artifact@cf430e030ddbb5b0abf93d22962f4752f3646cd9 # v7.0.2
        if: always()
        with:
          name: rapport-tests
          path: ci-reports/
          if-no-files-found: ignore

  build-image:
    name: Build de l'image Docker
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
      - name: Construire l'image
        run: docker build -t notes-app:${{ github.sha }} .
      - name: Exporter l'image (meme artefact pour le DAST et la release)
        run: docker save notes-app:${{ github.sha }} | gzip > notes-app-image.tar.gz
      - uses: actions/upload-artifact@cf430e030ddbb5b0abf93d22962f4752f3646cd9 # v7.0.2
        with:
          name: notes-app-image
          path: notes-app-image.tar.gz
          retention-days: 1
          compression-level: 0

  # ------------------------------------------------------------------- phase 2
  sast:
    name: SAST - Bandit
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
      - uses: actions/setup-python@6d4fd22b20465ccc0f7354476588e8df7c9ca0e0 # v7.1.0
        with:
          python-version: ${{ env.PYTHON_VERSION }}
          cache: pip
          cache-dependency-path: requirements*.txt
      - name: Installer Bandit
        run: python -m pip install -r requirements-dev.txt
      - name: Rapports complets (tous niveaux, non bloquants)
        run: |
          mkdir -p ci-reports
          bandit -r app -f json -o ci-reports/bandit.json --exit-zero
          bandit -r app -f html -o ci-reports/bandit.html --exit-zero
          python scripts/security_report.py bandit ci-reports/bandit.json
      - name: "GATE SAST : echec si une vulnerabilite de severite HIGH est detectee"
        run: bandit -r app --severity-level "$BANDIT_FAIL_LEVEL"
      - uses: actions/upload-artifact@cf430e030ddbb5b0abf93d22962f4752f3646cd9 # v7.0.2
        if: always()
        with:
          name: rapport-sast-bandit
          path: ci-reports/
          if-no-files-found: ignore

  sca-pip-audit:
    name: SCA - pip-audit
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
      - uses: actions/setup-python@6d4fd22b20465ccc0f7354476588e8df7c9ca0e0 # v7.1.0
```

```
        with:
          python-version: ${{ env.PYTHON_VERSION }}
          cache: pip
          cache-dependency-path: requirements*.txt
      - name: Installer pip-audit
        run: python -m pip install -r requirements-dev.txt
      - name: Rapport JSON et resume (non bloquants)
        run: |
          mkdir -p ci-reports
          pip-audit -r requirements.txt --no-deps --disable-pip -f json -o ci-reports/pip
  -audit.json || true
          python scripts/security_report.py pip-audit ci-reports/pip-audit.json
      # pip-audit n'expose pas de score CVSS (base PyPA/OSV) : toute vulnerabilite
      # connue de requirements.txt bloque, un correctif existant etant toujours indique.
      - name: "GATE SCA (1/2) : echec si une dependance a une vulnerabilite connue"
        run: pip-audit -r requirements.txt --no-deps --disable-pip --desc off
      - uses: actions/upload-artifact@cf430e030ddbb5b0abf93d22962f4752f3646cd9 # v7.0.2
        if: always()
        with:
          name: rapport-sca-pip-audit
          path: ci-reports/
          if-no-files-found: ignore

  sca-dependency-check:
    name: SCA - OWASP Dependency-Check
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
      - uses: actions/setup-python@6d4fd22b20465ccc0f7354476588e8df7c9ca0e0 # v7.1.0
        with:
          python-version: ${{ env.PYTHON_VERSION }}
      - name: Cle de cache hebdomadaire de la base NVD
        id: nvd
        run: echo "week=$(date -u +%G-W%V)" >> "$GITHUB_OUTPUT"
      - name: Cache de la base NVD
        uses: actions/cache@55cc8345863c7cc4c66a329aec7e433d2d1c52a9 # v6.1.0
        with:
          path: ~/dc-data
          key: dc-data-${{ steps.nvd.outputs.week }}
          restore-keys: dc-data-
      # --enableExperimental : l'analyseur pip de requirements.txt est experimental
      # --nvdDatafeed        : miroir officiel de la base NVD (evite l'API NVD, tres
    lente sans cle)
      # --failOnCVSS 7       : code de sortie 15 si une CVE de score >= 7 (High/Critical)
     est trouvee
      - name: "GATE SCA (2/2) : OWASP Dependency-Check, echec si CVSS >= 7"
        run: |
          mkdir -p ~/dc-data ci-reports/dependency-check
          chmod -R 777 ~/dc-data ci-reports/dependency-check
          docker run --rm \
            -v "$PWD:/src:ro" \
            -v "$HOME/dc-data:/usr/share/dependency-check/data" \
            -v "$PWD/ci-reports/dependency-check:/report" \
            owasp/dependency-check:13.0.0 \
            --project "TP2 Notes App" --scan /src \
            --exclude "**/.git/**" --exclude "**/ci-reports/**" \
            --enableExperimental --disableOssIndex \
            --nvdDatafeed "https://dependency-check.github.io/DependencyCheck_Builder/
    nvd_cache/" \
            --format HTML --format JSON --out /report \
            --failOnCVSS "$DC_FAIL_ON_CVSS"
      - name: Resume du rapport Dependency-Check
        if: always()
        run: python scripts/security_report.py dependency-check ci-reports/dependency-
    check/dependency-check-report.json
      - uses: actions/upload-artifact@cf430e030ddbb5b0abf93d22962f4752f3646cd9 # v7.0.2
        if: always()
        with:
          name: rapport-sca-dependency-check
          path: ci-reports/
          if-no-files-found: ignore
```

```
# ------------------------------------------------------------------ phase 3
dast:
  name: DAST - OWASP ZAP (environnement ephemere)
  runs-on: ubuntu-latest
  needs: build-image
  timeout-minutes: 30
  env:
    IMAGE: notes-app:${{ github.sha }}
    OUT_DIR: ci-reports/zap
    FAIL_ON: high              # niveau d'alerte ZAP qui bloque (ZAP n'a pas de niveau
  "critical" : High est le plus eleve)
  steps:
    - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
    - uses: actions/download-artifact@9000827ccba6bdab643e8b6fd33ac0654aef8333 # v8.0.2
      with:
        name: notes-app-image
    - name: Charger l'image construite par le job build-image
      run: docker load --input notes-app-image.tar.gz
    - name: Deployer l'environnement ephemere (reseau Docker + application)
      run: bash scripts/dast.sh deploy
    - name: Scan DAST OWASP ZAP (spider + scan actif authentifies)
      run: bash scripts/dast.sh scan
    - name: "GATE DAST : echec si une alerte de risque High est detectee"
      run: bash scripts/dast.sh gate
    - name: Detruire l'environnement ephemere
      if: always()
      run: bash scripts/dast.sh destroy
    - uses: actions/upload-artifact@cf430e030ddbb5b0abf93d22962f4752f3646cd9 # v7.0.2
      if: always()
      with:
        name: rapport-dast-zap
        path: ci-reports/zap/
        if-no-files-found: ignore

# Feu vert unique : c'est le seul controle exige par la protection de la branche main.
security-gate:
  name: Security gate
  runs-on: ubuntu-latest
  needs: [build-test, build-image, sast, sca-pip-audit, sca-dependency-check, dast]
  if: always()
  steps:
    - name: Bilan des controles
      env:
        NEEDS: ${{ toJSON(needs) }}
      run: |
        {
          echo "### Security gate"
          echo
          echo "| Controle | Resultat |"
          echo "|---|---|"
          echo "$NEEDS" | jq -r 'to_entries[] | "| \(.key) | \(.value.result) |"'
        } | tee -a "$GITHUB_STEP_SUMMARY"
        if echo "$NEEDS" | jq -e 'to_entries | any(.value.result != "success")' > /dev/
  null; then
          echo "::error::Au moins un controle n'est pas vert : version non securisee,
  publication bloquee."
          exit 1
        fi
        echo "Tous les controles sont verts : la publication est autorisee."

# Publication : uniquement sur main, uniquement si le security gate est vert, et
# c'est l'image testee par le DAST (meme artefact) qui est publiee.
release:
  name: Release (publication de l'image)
  runs-on: ubuntu-latest
  needs: security-gate
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  timeout-minutes: 10
  permissions:
    contents: read
    packages: write
  steps:
    - uses: actions/download-artifact@9000827ccba6bdab643e8b6fd33ac0654aef8333 # v8.0.2
```

```
        with:
          name: notes-app-image
      - name: Charger l'image validee
        run: docker load --input notes-app-image.tar.gz
      - name: Publier l'image sur GitHub Container Registry
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: |
          REGISTRY_IMAGE="ghcr.io/${GITHUB_REPOSITORY_OWNER,,}/notes-app"
          echo "$GITHUB_TOKEN" | docker login ghcr.io -u "$GITHUB_ACTOR" --password-stdin
          docker tag "notes-app:${GITHUB_SHA}" "${REGISTRY_IMAGE}:${GITHUB_SHA}"
          docker tag "notes-app:${GITHUB_SHA}" "${REGISTRY_IMAGE}:latest"
          docker push "${REGISTRY_IMAGE}:${GITHUB_SHA}"
          docker push "${REGISTRY_IMAGE}:latest"
          echo "Image publiee : ${REGISTRY_IMAGE}:${GITHUB_SHA}" | tee -a "
    $GITHUB_STEP_SUMMARY"
```

Annexe B – Plan ZAP `zap/automation.yaml`

Listing 10 – Plan d'automatisation ZAP

```
---
# Plan OWASP ZAP (Automation Framework) : scan DAST authentifie de la Notes App.
#
# Lance par scripts/dast.sh dans un conteneur ZAP raccorde au reseau Docker de
# l'environnement ephemere : l'application y est joignable sous http://app:5000.
# Variables d'environnement attendues : ZAP_USER et ZAP_PASSWORD (compte jetable
# cree dans l'environnement ephemere, jamais ecrit dans le depot).
env:
  contexts:
    - name: notes-app
      urls:
        - http://app:5000
      includePaths:
        - http://app:5000.*
      excludePaths:
        # Evite que le scanner se deconnecte en suivant le lien "Deconnexion"
        - http://app:5000/logout.*
      authentication:
        method: form
        parameters:
          loginPageUrl: http://app:5000/login
          loginRequestUrl: http://app:5000/login
          loginRequestBody: username={%username%}&password={%password%}
        verification:
          method: response
          # Le lien "Deconnexion" n'apparait que lorsque la session est ouverte
          loggedInRegex: "\\QDeconnexion\\E"
          loggedOutRegex: "\\QSe connecter\\E"
      sessionManagement:
        method: cookie
      users:
        - name: tester
          credentials:
            username: ${ZAP_USER}
            password: ${ZAP_PASSWORD}
  parameters:
    failOnError: true
    failOnWarning: false
    progressToStdout: true

jobs:
  - type: passiveScan-config
    parameters:
      maxAlertsPerRule: 20

  # Triage d'un faux positif (voir RAPPORT.md, section 5.6).
  # La regle "SQL Injection" (40018, detection booleenne par comparaison de pages) s'est
  # declenchee une fois sur POST /preferences (parametre theme) alors que le code n'avait
```

```
  # pas change : ce parametre n'atteint jamais SQL (il est compare a une liste blanche
    puis
  # range dans le cookie de session) et la page affichee depend de l'etat de la session,
    que
  # les requetes concurrentes du scanner modifient entre elles.
  # Le filtre est volontairement etroit (regle + URL + parametre + methode) : la regle
    reste
  # active partout ailleurs. Son hypothese est verrouillee par le test
  # tests/test_app.py::test_preferences_never_touch_the_database.
  - type: alertFilter
    alertFilters:
      - ruleId: 40018
        newRisk: False Positive
        context: notes-app
        url: http://app:5000/preferences
        parameter: theme
        methods:
          - POST

  # Routes exposees par l'application mais non liees depuis l'interface HTML :
  # on les declare pour que le scan actif les attaque aussi.
  - type: requestor
    parameters:
      user: tester
    requests:
      - url: http://app:5000/tools/ping?host=127.0.0.1
        method: GET
      - url: http://app:5000/tools/fetch?url=https://example.com
        method: GET

  - type: spider
    parameters:
      context: notes-app
      user: tester
      maxDuration: 3
      maxDepth: 5
    tests:
      # Garde-fou : si l'authentification echoue, le spider ne voit que /login et
      # /register. Le job echoue alors au lieu de "valider" un scan presque vide.
      - name: Le spider explore bien l'application authentifiee
        type: stats
        statistic: automation.spider.urls.added
        operator: ">="
        value: 8
        onFail: error

  - type: passiveScan-wait
    parameters:
      maxDuration: 5

  - type: activeScan
    parameters:
      context: notes-app
      user: tester
      maxScanDurationInMins: 12
    policyDefinition:
      defaultStrength: medium
      defaultThreshold: medium

  - type: report
    parameters:
      template: traditional-json
      reportDir: /zap/wrk
      reportFile: zap-report
      reportTitle: TP2 DevSecOps - scan DAST de la Notes App
  - type: report
    parameters:
      template: traditional-html
      reportDir: /zap/wrk
      reportFile: zap-report
      reportTitle: TP2 DevSecOps - scan DAST de la Notes App
```

Annexe C – Index des preuves d'exécution

Les rapports et journaux des exécutions de référence sont conservés dans le dépôt, dossier `reports/` (les artefacts GitHub expirent après 90 jours).

Dossier	**Exécution**	**Contenu**
`pr1-vulnerable/`	#5, PR n° 1 (`0285a71`)	Pipeline en échec : Bandit, pip-audit, Dependency-Check, ZAP, journaux des étapes en échec
`pr2-fixed/`	#6, PR n° 2 (`c93889e`)	Pipeline vert : mêmes rapports, sans alerte bloquante
`pr2-faux-positif/`	#10, PR n° 2 (`fe474eb`)	DAST bloqué par le faux positif ZAP
`main-release/`	#3, *push* sur `main` (`2b88fac`)	Résultat des huit jobs et journal de la publication

Les figures de ce compte rendu proviennent de ces exécutions : pages GitHub du dépôt (`t_gh_*`), rapports HTML des outils (`r_*`), journaux des étapes (`t_log_*`) et application exécutée en local (`a_*`).